IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 14 October 2025, accepted 25 November 2025, date of publication 1 December 2025,
date of current version 5 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3639062

RESEARCH ARTICLE

Nonparametric Click Modeling Using Dirichlet Process Mixture Model for Information Retrieval

K. J. AMALA AND D. RAJESWARI

Department of Data Science and Business Systems, School of Computing, College of Engineering and Technology, SRM Institute of Science and Technology, Chengalpattu, Kattankulathur, Tamil Nadu 603203, India

Corresponding author: K. J. Amala (ak7858@srmist.edu.in)

ABSTRACT Click models are essential for comprehending user search behavior and enhancing ranking algorithms; nevertheless, current methodologies face challenges due to the variability of user interaction patterns across different query settings. Current techniques depend on static mixture components that might not accurately represent the diverse behavior of users across various contexts and demographics. This study presents a Dirichlet Process Mixture Model for click modeling that autonomously adjusts model complexity to accommodate diverse user behavior patterns without necessitating predetermined assumptions on the quantity of user behavior clusters. This approach develops an efficient inference algorithm that alternates between Bayesian cluster assignment and neural network training, enabling scalable learning for large-scale click prediction applications. The theoretical foundation builds upon established Dirichlet Process theory while extending it to neural click modeling, providing convergence guarantees for the proposed inference system. A comprehensive comparative evaluation of click modeling approaches is conducted, comparing the proposed method to five established baseline techniques across nine distinct click model configurations. The experimental results indicate that the Dirichlet Process Mixture Model consistently excels existing baselines across various evaluation metrics and demonstrates a particular aptitude for rating quality metrics when contrasted with the baseline averages. Experimental validation on real-world data shows that Dirichlet Process Mixture Model achieves substantial improvements over existing methods, with a 75.5% relative improvement in Mean Average Precision (0.6138 vs. 0.3496) and 48.7% improvement in Precision@1 (0.6539 vs. 0.4490).

INDEX TERMS Click model, Dirichlet process mixture model, click-through rate, unbiased learning to rank, two-tower model, ranking.

I. INTRODUCTION

Learning to rank (LTR) is a field that has evolved at the convergence of machine learning, information retrieval, and natural language processing so it involves utilizing machine learning approaches to execute ranking tasks. Practical applications of LTR include document retrieval, expert search, online advertising, recommender systems, question answering, key phrase extraction, document summarizing, and machine translation [1], [2].

Creating training data is one of the challenges in LTR issues, while document retrieval has become an example. To achieve the best possible results, the training data should be comprised of the ideal ranking lists of documents for each query. Click-through data is one of the methods of creating training data. Click-through data at a web search engine captures user clicks on documents subsequent to query submission and also it signifies implicit user feedback regarding relevance, making it valuable for relevance assessments. People tend to click on the documents at the top of a list, even if those documents are not quite what they seem to be. This means that documents near the top get more clicks. It is

The associate editor coordinating the review of this manuscript and approving it for publication was Joao Bernardo Ferreira Sequeiros.

called click bias when this happens [3]. Researchers have tried to eliminate the impact of bias in the training of ranking algorithms in order to fully utilize click data for LTR.

The creation of click models is one such endeavor [4]. Click models represent user search behavior as structured probabilistic processes using Bayesian networks. When a user examines a document and finds it appealing, only then they will click on it.

$Click_d = 1 \iff Exam_d = 1$ and $Attr_d = 1$, where $Click_d$ indicate user clicked on document d, $Exam_d$ for user examined the document d (depends on position), $Attr_d$ indicates user is attracted to the document d (depends on relevance/snippet). So the chance of a click is modeled as

$P(Click_d = 1) = P(Exam_d = 1) \cdot P(Attr_d = 1)$

By optimizing the likelihood of the observed user clicks, click models estimate the true (unbiased) relevance feedback based on predictions about how people look at the content on a Search Engine Result Page (SERP). After that, ranking models are trained using the predicted relevance signals in order to ensure that the system as a whole is unbiased. Click models make different assumptions about how people will interact with a web search engine return page and can simulate user interactions when real click data is limited. While effective in many settings, these models rely on handcrafted assumptions about user behavior. In practice, users exhibit heterogeneous and dynamic behaviors that may not align with any single predefined model. According to the work proposed in [5], click models are generally divided into probability graph-based click models, neural network-based click models, and hybrid click models, and each one possesses its own specialities.

The limitations of traditional probabilistic approaches motivated the adoption of neural architectures for click prediction, with two-tower models emerging as a particularly influential paradigm [6]. Two-tower models are the modern instantiation of the classic Position Based Model (PBM) commonly used for ranking tasks and widely used to remove bias in the click data. There are two neural networks: the first one takes relevance features that quantify how relevant the current document might be for the current query, and the other one is called the bias tower, where it uses features that might influence the examination of an item. Then the two towers are combined in different ways such as multiplying two probabilities or additive or dot products, and in the end, they try to predict the click. But recent studies reported a concerning fact that training two-tower models on clicks collected by well-performing production systems leads to decreased ranking performance [7]. Despite the representational advances of neural architectures, both traditional probabilistic models and modern neural approaches share a critical limitation: the assumption of homogeneous user behavior. This assumption manifests in the use of global model parameters that apply uniformly across all users and sessions, despite substantial empirical evidence of behavioral heterogeneity in search tasks.

There are a number of existing approaches to click prediction that try to account for user heterogeneity, but they all have major drawbacks:

a) Techniques for User Segmentation: Approaches in [8] and [9] try to represent heterogeneity by manually segmenting users according to observable traits like demographics, session duration, or query frequency. On the other hand, these methods could miss the mark when it comes to identifying the real patterns of behavior that motivate clicks and necessitate domain knowledge to create useful segments. In contrast, our DPMM framework uses demographic and contextual features as inputs to neural components rather than as criteria for segmentation. Specifically, features are fed into the two-tower architecture, where their predictive importance is learned from data.

b) Approaches to Customization: In order to account for unique characteristics, neural networks have included user embeddings and customization layers [10]. Although these methods work well for known users with sufficient interaction history, they rely on user-specific parameters (e.g., user-ID embeddings) that do not exist for new users and fail to discover population-level behavioral patterns that generalize across users. In contrast, our method learns shared behavioral components that apply to multiple users exhibiting similar patterns. Through probabilistic component assignment, new users are assigned to existing behavioral patterns discovered from the population, enabling predictions without requiring user-specific parameter learning. This architectural choice provides robustness when generalizing to users not seen during training, though we emphasize that our primary contribution is modeling behavioral heterogeneity rather than specifically addressing the cold-start problem.

c) Multi-Task Learning: Certain methodologies utilize multi-task learning frameworks [11] to concurrently model many facets of user behavior. Nevertheless, these methods necessitate the pre-definition of task parameters and fail to autonomously identify the inherent behavioral clusters within the data. But in our framework, Gibbs sampling creates new components when data points are poorly explained by existing patterns, removes empty components automatically, and stabilizes at the optimal K* that balances data fit and model parsimony.

The primary constraint inherent in all these methodologies is the necessity for prior definition of user behavior patterns. More clearly, there is a need for algorithms that can detect hidden patterns in user behavior without having to specify categories of activity in advance.

This work addresses behavioral heterogeneity in click prediction by building upon the theoretical foundations of Dirichlet Process Mixture Models (DPMM) established by Ferguson [12] and Antoniak [13]. This work proposes Neural Dirichlet Process Mixture Models (Neural DPMM), representing the first application of DPMM to click modeling for information retrieval tasks. Each behavioral cluster is represented by a specialized neural network that learns

cluster-specific patterns in user click behavior. Rather than proposing architectural modifications to existing models, this work conducts a comprehensive comparison study evaluating Neural DPMM against five state-of-the-art click modeling methods, assessing algorithmic effectiveness across diverse user behavior scenarios.

The proposed research provides multiple substantial contributions to the field of click prediction.

- Methodological Innovation: Proposed research introduces Neural Dirichlet Process Mixture Models for click prediction, combining Bayesian clustering with neural architectures to automatically discover user behavioral patterns without predetermined cluster numbers.
- Algorithmic Development: This work develops an efficient inference algorithm with theoretical convergence guarantees that alternates between Bayesian cluster assignment and neural network training for scalable click modeling.
- Empirical Validation: This work demonstrates superior performance compared to established click models and neural baselines across multiple datasets, providing interpretable insights into discovered behavioral patterns.

II. BACKGROUND

A. CLICK THROUGH RATE MODELS

Personalized LTR has been investigated in extensive industrial contexts, such as Airbnb's dual-tower neural architecture [14], which concurrently learns user and item representations to encapsulate user-specific preferences. These methods show how useful it is to explicitly model user-document interactions, which works well with bias-correction methods like click models and mixture modeling approaches. But it does not deeply capture session dynamics or short-term shifts in user intent during a search session and this method relies heavily on historical behavioral data (bookings, clicks). For new users (no history) or new listings, personalization is limited, and the embeddings may not be reliable. DSIN [15] explicitly models user histories as sessions, utilizing multi-head self-attention, bias encoding, and Bi-LSTM layers to capture dependencies both within and between sessions. This session-aware modeling helps DSIN better understand the different interests of users and make better predictions about Click Through Rate (CTR).

Deep Multi-Interest Network (DMIN) [16] was suggested as a way to better predict CTR by directly modeling multiple user interests at the same time. DMIN adds a behavior refiner layer that uses multi-head self-attention with auxiliary loss to improve representations of user behavior. This is different from earlier models like DIN [17] and DIEN [18], which assume a single main interest or only focus on sequential evolution. Then, a multi-interest extractor layer is used to find a range of hidden interests. These are triggered dynamically based on the target item and then combined with features from the user profile, the context, and the item in order to make a prediction. Even with these pros, DMIN does have some cons. There is a fixed number of attention heads that affects the number of latent interests. This makes it hard to adapt to users whose behavior is more complicated than others. The model also relies on detailed sequential records to work well and it may not work as well when there is not enough data or when the system starts up cold. Also, the design takes into account different types of interests for predicting CTR, but it does not directly take into account how preferences change over time or other business goals like conversion and retention. When DSIN [15] is used, user behavior sequences are divided into sessions based on a predefined rule of thirty minutes of idleness. This heuristic may not always accurately reflect the bounds of the session, and it may incorrectly identify changes in the user's intent.

By explicitly modeling the hierarchical structure of user interests, Deep Hierarchical Interest Network (DHIN) [19] was presented as a means of improving the prediction of CTR. Individual user actions are initially encoded as low-level behavior embeddings, which are then aggregated through self-attention into higher-level latent intent embeddings. DHIN introduces a two-level representation. A hierarchical interest evolution module is responsible for capturing both the short-term and long-term dynamics of these intentions. For the purpose of predicting click probabilities, the representations that are produced as a result of this module are merged with target item embeddings. Experiments conducted on large-scale industrial datasets demonstrate that DHIN routinely outperforms strong baselines, particularly in situations where user interests are diverse and multi-level. However, the strategy does have a few drawbacks to worry about. It is possible for real-time deployment to be negatively impacted by the increased computing overhead that is caused by the reliance on hierarchical self-attention and several encoding layers. Additionally, in order to construct meaningful intent hierarchies, the model necessitates extensive behavioral histories, which restricts its usefulness in situations when there is a lack of data or a cold start.

To overcome the shortcomings of sequential models which presume a single dominant intent per session, Deep Session Heterogeneity-Aware Network (DSHN) [20] was suggested as a solution to enhance CTR prediction. DSHN uses self-attention and a gating mechanism to dynamically weight heterogeneous behaviors, modeling user sessions as a blend of multiple latent interests. But it could not work as well in sparse data situations since it needs large-scale behavioral data to learn diverse interests. The learnt latent intent representations have a low level of interpretability, which makes predictions less transparent. In real life, users often click in different ways based on their own preferences, the reason for their search is to find information or to navigate or even just because of loud interactions. A two-component decomposition is also in Position Bias Aware Learning framework (PAL) [21], which is proposed for CTR prediction in a live recommender system, capable of modeling position bias during offline training and performing online inference

without position data. PAL is expressly engineered to rectify position bias; however, it is incapable of mitigating other significant biases.

Neural Click Model (NCM) [22] was introduced as a deep learning based alternative to traditional probabilistic click models. Instead of relying on handcrafted assumptions about examination and relevance, NCM employs recurrent neural networks (RNNs) to directly capture sequential dependencies in user click behavior across ranked lists. Given query–document features, position information, and previous clicks, the model predicts the click probability at each position in an end-to-end manner. Experiments on large-scale search engine logs demonstrated that NCM outperforms classical click models in terms of perplexity and NDCG, establishing the potential of neural architectures for click modeling. Despite these advantages, NCM has limitations. The reliance on RNNs increases computational cost, making real-time deployment more challenging compared to lightweight probabilistic models. The approach also requires large-scale click data for effective training, limiting its applicability in low-data or cold-start scenarios. Furthermore, while NCM is more flexible than traditional models, it sacrifices interpretability, as the learned neural representations do not provide clear insights into examination or relevance biases.

Two common ways to develop rankers from user clicks that are affected by position bias are inverse-propensity scoring and neural click models [23]. In this study, pointwise learning framework is used to examine the theoretical distinctions. It concludes that neural click models might be affected by position bias when learning from shared, sometimes conflicting, features instead of treating each document separately.

The work proposed in [24] employs Online Dependent Click Model within Hybrid Online–Offline LTR techniques using SAS-Rank and ES-Rank. It takes input as query-document features and outputs a linear ranking function. After the experimentation, the result shows they have improved predictive NDCG and MAP on LETOR datasets. The work proposed in [25] reexamines the concept of click model on the basis of mathematical relations between observable variables. It introduced three key design choices such as global dependencies, sequentiality, and factorization. It provides a comprehensive foundation for extending click modeling to new interaction paradigms beyond traditional ranked lists.

B. FINITE MIXTURE MODEL

Mixture model [26] posits that the clicks originate from several types of user actions. Each user/session may exhibit distinct click behavior: Certain users prioritize position, others prioritize content, and some exhibit behavior akin to PBM or even cascade models among others. Rather than selecting a singular model, mixture models acquire a blend (or mixture) of these models. A mixture model indicates that data comes from a mix of different patterns or distributions, each of which stands for a different underlying cause (or latent component). Here, employ several two-tower models, each representing distinct user behaviors, and utilize a mixing model to adaptively integrate them for each session or user.

Mathematically:

$$P(click \mid query, doc,\ k) = \sum_{z=1}^{K} p_z \cdot P^{(z)}_{\text{click}}(query, doc, k)$$

- p_z: mixing weight of the z-th click behavior (mixture component), i.e., how likely that user behavior is overall.
- $P^{(z)}_{\text{click}}(query, doc, k)$: click probability under the z-th click model, based on query *query*, document *doc*, and position k.
- K: total number of mixture components, i.e., different user behavior patterns.

Overall click probability is a weighted average over several possible user behaviors, where each behavior model gives a different probability of clicking based on *query*, *doc*, k.

Mixture model [26] has also added user-based Expectation Maximization(EM) method and embedding interaction strategies to two-tower models on fixed mixture structures and established behavior patterns for Unbiased LTR (ULTR).

Dirichlet Process Mixture Model framework [12], [27] provides a principled approach for automatic model complexity selection through bayesian nonparametric methods. While DPMM has shown success in clustering and topic modeling applications [28], its potential for click behavior modeling remains unexplored. This study adds a Dirichlet Process Mixture Model (DPMM) to the two-tower structure to find hidden patterns in user click behavior without having to specify the type of clicks ahead of time. This nonparametric method lets the model directly capture a wide range of user interactions, even if they are noisy, like query-dependent examination patterns or position-relevance entanglement. Here keep the two-tower architecture's ability to grow and be broken down into smaller parts, but it also makes it far more powerful.

III. METHODOLOGY

A. DIRICHLET PROCESS MIXTURE MODEL (DPMM)

Traditional click prediction models assume a single, homogeneous user behavior pattern across all search contexts. In reality, users exhibit diverse behavioral modes. Current approach adapts the DPMM framework [29], [30] to the click modeling domain, extending the traditional clustering formulation to handle sequential user interaction patterns. In contrast to finite mixture models, DPMM offers the adaptability to modify the number of mixture components in response to increasing data complexity, thus preventing underfitting due to insufficient components or overfitting from excessively large ones. Following the DPMM framework [31], this approach adapts the infinite mixture modeling paradigm to capture heterogeneous user click behaviors without requiring prior specification of the number of behavior clusters.

The full working pipeline is written in Algorithm 1 named DPMM overall workflow in Supplementary Material. Current methodology integrates three critical innovations:

a) Automatic Pattern Discovery: Instead of establishing a fixed number of behavioral components K, this model employs the Chinese Restaurant Process (CRP) to dynamically determine the optimal number K* during training. The model initiates with a single component and generates new behavioral patterns as required.

b) Specialized Component Architecture: A two-tower neural network is used to characterize each behavioral pattern that has been discovered.

c) Assignment Based on Probability: Data points are assigned to behavioral components using a principled bayesian approach.

B. PROBLEM FORMULATION

Given a dataset of search interactions consisting of query-document features x_{qd}, position feature x_{pos}, binary click labels $y \in \{0, 1\}$. The goal is to learn a mixture model that automatically discovers K^* behavioral components and their parameters $\{f_k\}_{k=1}^{K^*}$ without pre-specifying K^*.

1) TRADITIONAL FINITE MIXTURE LIMITATION

Conventional mixture models require the number of components K to be specified in advance. Formally, such models are parameterized by a vector of mixing proportions $p = (p_1, \ldots, p_K)$ and $\sum_j p_j = 1$ and a set of component-specific parameters f_k for each of K components. Each user/click belongs to one of these K patterns with probability determined by p. Despite the fact that it is effective in controlled situations, this formulation has a significant limitation such that it assumes the number of behavioral patterns in the data. In practice, user interaction data often exhibit unknown and evolving heterogeneity. Choosing K incorrectly can result in underfitting (if K is too small) or overfitting (if K is too large). Moreover, the need to train and compare models for multiple candidate values of K introduces substantial computational overhead, making finite mixture approaches both inefficient and brittle in dynamic, real-world settings.

2) DIRICHLET PROCESS SOLUTION

To overcome the rigidity of finite mixture models, proposed method adopts a Dirichlet Process (DP) prior, which allows the model to flexibly infer the appropriate number of behavioral patterns directly from the data. Rather than fixing K in advance, the DP defines a distribution over an unbounded number of potential components:

$$G \sim \mathrm{DP}(\theta, H),$$

where θ is concentration parameter (controls expected number of new patterns), H is base distribution (prior over behavioral patterns), G is random distribution over patterns (learned from data). The base distribution encodes prior beliefs about the structure of behavioral patterns, while the random measure G represents the data-driven distribution over components that emerges during training. The concentration parameter θ plays a critical role: small values of θ bias the model toward fewer, broader components, while larger values promote the discovery of many specialized clusters. Importantly, the Dirichlet Process framework ensures that the number of active components grows adaptively with data complexity, thereby eliminating the need for manual specification of K and enabling principled, data-driven discovery of heterogeneous behavioral modes.

Within the Dirichlet Process Mixture Model (DPMM) framework, each data point i has a latent component assignment, $z_i \in \{1, 2, \ldots\}$ and each component k has parameters f_k drawn from a base distribution H.

3) CHINESE RESTAURANT PROCESS: THE ASSIGNMENT MECHANISM

The Dirichlet Process methodology relies on the Chinese Restaurant Process (CRP) to intuitively allocate data points to behavioral patterns. Imagine a restaurant with an infinite number of tables, where each arriving customer (data point) must choose a table (behavioral component). The probability of joining an existing table is proportional to the number of customers already seated there, while the probability of starting a new table is proportional to the concentration parameter θ. Formally, for the i-th data point, the assignment probabilities are given by:

$$P(\text{join pattern } k) = \frac{n_k}{N - 1 + \theta}$$
$$P(\text{start new pattern}) = \frac{\theta}{N - 1 + \theta}$$

where n_k denotes the number of data points currently assigned to pattern k, and N is the total number of data points observed so far.

This mechanism has several key properties. First, it exhibits a ''rich get richer'' effect, whereby popular patterns attract more data points, leading to naturally balanced yet interpretable clustering. Second, the process always allows for the emergence of new behavioral patterns, with the likelihood controlled by θ. Third, there is no predetermined upper bound on the number of clusters, ensuring flexibility in capturing heterogeneous behaviors. Importantly, the expected number of active patterns grows only logarithmically with the dataset size,

$$\mathbb{E}[K_n] \approx \theta \log n$$

which guarantees scalability while preserving the capacity to model increasing complexity in user interactions.

4) LIKELIHOOD-WEIGHTED ASSIGNMENT

While the Chinese Restaurant Process (CRP) provides a prior mechanism for assigning data points to behavioral components, it considers only popularity effects and ignores how well a component explains the observed data. To address this limitation, the likelihood of user behavior is incorporated

into the assignment probabilities. Specifically, the probability of assigning user i to component k is given by:

$$P(z_i = k \mid z_{-i}, x_i, y_i) \propto \frac{n_k}{N-1+\theta} \times L(y_i \mid x_i, f_k)$$

where the likelihood term $L(y_i \mid x_i, f_k)$ measures how well component k's neural network explains the observed click outcome:

$$L(y_i \mid x_i, f_k) = \left(f_{f_k}(x_i)\right)^{y_i} \left(1 - f_{f_k}(x_i)\right)^{1-y_i}$$

Here, $f_{f_k}(x_i)$ denotes the predicted click probability from component k. This formulation ensures that assignments balance pattern popularity (from CRP) with explanatory power (from the neural network), enabling the model to favor components that both fit the data and represent meaningful behavioral structures. Importantly, this likelihood-based assignment naturally induces user-specific component memberships: users exhibiting similar behavioral patterns will have high assignment probabilities to the same components, while users with distinct behaviors will be assigned to different components. This data-driven clustering enables the model to capture heterogeneous user behaviors without requiring manual segmentation or separate model training for each user group.

5) MODELING USER HETEROGENEITY VIA PERSONALIZED COMPONENT MIXING

Traditional click models assume homogeneous user behavior by applying a single set of global parameters θ_{global} uniformly to all users:

$$\hat{y}_i = f(x_i; \theta_{\text{global}}) \quad \text{for all users } i$$

This approach fails to capture behavioral diversity, as position-biased users and relevance-focused users are modeled identically.

Our DPMM framework addresses this limitation through a fundamentally different approach: rather than learning one global behavioral model, we discover K* distinct behavioral components $\{\theta_1, \theta_2, \ldots, \theta_{K*}\}$, each specializing in a different interaction pattern. Critically, different users are assigned to different combinations of these components based on their observed behavior.

User-Specific Predictions: For a new user with features x, the prediction is computed as a personalized weighted ensemble:

$$\hat{y}_i = \sum_{k=1}^{K^*} P(z_i = k \mid \text{data}_i) \cdot f_{\theta_k}(x_i)$$

where $P(z_i = k|data_i)$ are user-specific mixture weights determined by how well each component explains that particular user's click patterns.

Key distinction: While component parameters θ_k are shared across users within the same behavioral pattern, the mixture weights $P(z_i = k)$ are personalized to each user through the CRP assignment mechanism. This achieves heterogeneity without requiring separate parameters for each user.

For example, consider two users:

- **User A(position-biased):** $P(z_A = k_{\text{position}}) = 0.85, \quad P(z_A = k_{\text{relevance}}) = 0.15$
- **User B(relevance-focused):** $P(z_B = k_{\text{position}}) = 0.12, \quad P(z_B = k_{\text{relevance}}) = 0.88$

Even though both users use the same component parameters $\theta_{position}, \theta_{relevance}$, their predictions differ substantially due to personalized mixing weights, effectively creating user-specific models without the overfitting risks of truly individualized parameters.

This mixture-based approach lies between two extremes:

- **Fully global:** One model for all users → no heterogeneity
- **Fully personalized:** One model per user → data inefficiency, overfitting
- **Our DPMM:** $K*$ shared components + personalized mixing → balanced approach

C. COMPONENT ARCHITECTURE: TWO-TOWER NEURAL NETWORKS

Each behavioral component k is implemented as a two-tower neural network that separately captures relevance and positional effects before combining them into a unified click prediction. For a given input feature vector,

$$x_i = [\, x_{qd,i} \; ; \; x_{pos,i} \,]$$

where $x_{qd,i}$ is query-document relevance features and $x_{pos,i}$ represent position and contextual features. This specialized architecture allows each component to model different types of behavioral dependencies such as relevance-driven clicks, position-biased clicks, or hybrid patterns where both factors interact. By assigning users to different neural network components, the model naturally discovers and disentangles distinct behavioral modes without requiring manual specification.

Design Rationale: The allocation of features to specific towers reflects computational efficiency and prior knowledge about typical click behavior patterns, rather than rigid behavioral assumptions. We assign positional information and contextual features (including demographics) to the position tower based on observations from prior work [8], [9]. However, this architectural choice serves as an inductive bias rather than a constraint. The model retains full flexibility to learn that demographic or contextual features are irrelevant for certain behavioral components or that they interact with relevance signals in complex ways through the bilinear interaction layer.

1) TWO-TOWER PROCESSING

Each behavioral component k is modeled by a two-tower neural network, designed to separately capture query-document relevance and position-contextual patterns, and then integrate them through a nonlinear interaction layer.

(a) Relevance Tower: Processes query-document features x_{qd} through a two-layer neural network to capture the semantic relevance between queries and documents. This tower learns representations that encode content-based matching signals, such as term overlap, semantic similarity, and topical relevance. Hidden representation from the relevance tower for behavioral component k is calculated as

$$h_{\text{rel}}(k) = \text{ReLU}\left(W_2(k) \cdot \text{ReLU}\left(W_1(k) \cdot x_{qd} + b_1(k)\right) + b_2(k)\right),$$

where $W_1(k)$, $W_2(k)$ are weight matrices and $b_1(k)$, $b_2(k)$ are bias vectors for relevance tower layers 1 and 2.

b) Position—Context Tower: Handles positional and contextual features x_{pos} through an independent neural pathway to model position-dependent click biases and contextual effects. This tower captures behavioral patterns, device-specific interactions, temporal browsing patterns, and user characteristics that influence clicking behavior independent of content relevance. Hidden representation from position context tower for behavioral component k calculated as

$$h_{\text{pos}}^{(k)} = \text{ReLU}\Big(W_4^{(k)} \cdot \text{ReLU}\big(W_3^{(k)} \cdot x_{\text{pos}} + b_3^{(k)}\big) + b_4^{(k)}\Big),$$

where $W_3(k)$, $W_4(k)$ are weight matrices and $b_3(k)$, $b_4(k)$ are bias vectors for position-context tower layers 3 and 4.

c) Interaction Layer: The outputs of the two towers are then combined in an interaction layer that models higher-order dependencies between relevance and position-contextual signals. The predicted click probability for component k is:

$$f_{\theta_k}(x_i) = \sigma\Big(\big(h_{\text{rel}}^{(k)}\big)^\top U^{(k)} h_{\text{pos}}^{(k)} + W_5^{(k)}\big[h_{\text{rel}}^{(k)}; h_{\text{pos}}^{(k)}\big] + b_5^{(k)}\Big)$$

where $\sigma(\cdot)$ denotes the sigmoid activation, $[\cdot\,;\,\cdot]$ represents feature concatenation, $U^{(k)}$ denotes bilinear interaction matrix for component k that captures cross-dependencies between relevance and position representations, $W_5^{(k)}$ weight matrix for the concatenated tower outputs in component k, $b_5^{(k)}$ is bias vector for the final prediction layer in component k. This two-tower structure allows each component to independently learn relevance and position-context-specific representations, while the interaction layer captures their joint influence on click behavior. Different components thereby specialize in modeling distinct behavioral patterns. Crucially, these behavioral patterns are discovered from click data rather than assumed from demographic categories. The DPMM's likelihood-weighted assignment clusters users based on their observed clicking behavior given their features. This data-driven pattern discovery enables the model to capture behavioral heterogeneity that may or may not align with observable demographic boundaries, avoiding the limitations of manual segmentation while still leveraging rich contextual information.

D. TRAINING ALGORITHM

The DPMM training procedure alternates between Bayesian component assignment via Gibbs sampling and neural network parameter optimization. The algorithm automatically discovers the optimal number of behavioral components while learning their parameters through this iterative process.

Hyperparameter settings used in this part are as follows: θ: concentration parameter (θ =0.1), component training epochs: 100, η: Learning rate for Adam optimizer (0.001), : Numerical stability constant (10^-8), patience: Early stopping patience (20-epochs).

The detailed algorithm for training is written in Algorithm 2 named DPMM Training in Supplementary Material.

1) FINAL PREDICTION: WEIGHTED MIXTURE

Once the model has discovered its set of behavioral patterns, prediction for a new user interaction is made by combining the outputs of all active components. Specifically, the predicted click probability is:

$$\hat{y} = \sum_{k=1}^{K^*} w_k \cdot f_{\theta_k}(x), \quad \text{where } w_k = \frac{n_k}{N}.$$

where $f_{\theta_k}(x)$ is the prediction from component k's neural network, K^* is the number of active behavioral patterns discovered, w_k is the mixture weight assigned to component k defined as $w_k = \frac{n_k}{N}$ with n_k representing the number of users (or interactions) assigned to component k, and N being the total number of users.

Thus, each component contributes to the final prediction in proportion to how prevalent its behavioral pattern is in the data. This ensures that the model naturally balances common patterns with specialized behaviors, providing both interpretability and adaptability.

E. INFERENCE AND PREDICTION

Once training is completed, the model uses the discovered behavioral components to make predictions on new interactions. Inference in the proposed DPMM framework involves two stages:

i) Training-time inference – updating component assignments for data points using Gibbs sampling.

ii) Test-time prediction – computing final predictions as an ensemble of active components.

1) COMPONENT ASSIGNMENT PROBABILITIES (TRAINING-TIME INFERENCE)

For each training data point, the model infers its component assignment by balancing the likelihood under each component with the Chinese Restaurant Process (CRP) prior. For each training instance, the model evaluates two possibilities:

a: EXISTING COMPONENT PROBABILITY

The probability of assigning a data point to an existing component k combines two factors - the component's current popularity (how many other points are already assigned to it) and the likelihood that this component's learned parameters can explain the observed click behavior. Popular components with good explanatory power receive higher assignment

probabilities. It is represented as follows:

$$P\left(z_i = k \mid z_{-i},\, x^{(i)},\, y^{(i)}\right) \;\propto\; n_{k,-i} \;\times\; L\left(y^{(i)} \mid x^{(i)}, \theta_k\right),$$

where z_i is component assignment for data point i, z_{-i} indicate component assignments for all data points except instance i, k is the index of an existing behavioral component, $x^{(i)}$ is feature vector for the i-th training instance, $y^{(i)}$ is observed click label for the i-th training instance (0 = no click, 1 = click), $n_{k,-i}$ is the count of data points currently assigned to component k excluding instance i, $L\left(y^{(i)} \mid x^{(i)}, \theta_k\right)$ indicates the likelihood of observing click label given features under component k's parameters.

b: NEW COMPONENT PROBABILITY

Alternatively, the instance can initiate a new behavioral component. This probability is controlled by the concentration parameter θ and depends on how well default parameters can explain the data. Higher θ values encourage more component creation leading to finer-grained behavioral pattern discovery.

$$P\left(z_i = K^* + 1 \mid z_{-i},\, x^{(i)},\, y^{(i)}\right) \;\propto\; \theta \;\times\; L\left(y^{(i)} \mid x^{(i)}, \theta_0\right).$$

where θ the concentration parameter, θ_0 prior parameters for a new component, K^* is the current number of discovered behavioral components, and K^*+1 is the index for a potential new component. Detailed steps are written in the Algorithm 3 Component assignment update in Supplementary material.

2) GIBBS SAMPLING STEP

The Gibbs sampling step is the ''inference engine'' for component assignment. It keeps adjusting the cluster structure during training, deciding whether a user belongs to an existing pattern or whether a new pattern should be created.

3) LIKELIHOOD COMPUTATION

The likelihood computation measures how well each behavioral component's neural network explains the observed click data using binary cross-entropy loss. For clicked instances (y=1), the likelihood increases when the component predicts high click probability. For non-clicked instances (y=0), the likelihood increases when the component predicts low click probability.

$$\begin{aligned} L(y \mid x, \theta_k) &= \exp\Big(y \cdot \log\big(f_k(x)\big) \\ &\quad + (1-y) \cdot \log\big(1 - f_k(x)\big)\Big). \\ \log L(y \mid x, \theta_k) &= y \cdot \log\big(f_k(x) + \epsilon\big) \\ &\quad + (1-y) \cdot \log\big(1 - f_k(x) + \epsilon\big), \end{aligned}$$

where $\epsilon = 10^{-8}$ to prevent numerical instability when taking logarithms, $L(y \mid x, \theta_k)$: Likelihood of observing click label y given features x under component k's parameters, $f_k(x)$: Predicted click probability from component k's neural network, log-likelihood is computationally more stable than direct likelihood.

4) TEST-TIME PREDICTION (FORWARD INFERENCE)

Once training is completed, predictions for new data points are obtained through a weighted ensemble of discovered behavioral components. The final prediction combines individual component outputs proportional to their training data assignments:

Ensemble Prediction Formula:

$$\hat{y} = \sum_{k=1}^{K^*} w_k \cdot f_k(x),$$

where $K^* = |\{k : n_k > 0\}|$ is number of non-empty components. $w_k = \frac{n_k}{N}$ are mixture weights proportional to component size, $n_k = |\{i : z_i = k\}|$ is the number of training samples assigned to k, N = total number of training samples, $f_k(x) = \sigma\left(g_k(x_q, x_p)\right)$ is component k's prediction function.

$$\sum_{k=1}^{K^*} w_k = 1$$

It is called normalization property. Thus the component weights form a valid probability distribution, ensuring that the ensemble prediction is a convex combination of individual component predictions. Algorithmic representation is shown in Algorithm 4: DPMM Ensemble Prediction in Supplementary Material.

5) FORWARD PREDICTION

The prediction algorithm implements the ensemble formula from previous section, computing a weighted average of active component predictions where weights are proportional to component training data assignments.

6) COMPUTATIONAL COMPLEXITY

The computational cost of the main inference and prediction steps was examined. For Forward Prediction:

$$\mathcal{O}(K^* \cdot d)$$

since each of the K^* active components processes the feature vector.

Gibbs Sampling for assignment updates:

$$\mathcal{O}(N \cdot K^* \cdot d)$$

per iteration, because each of the$N$ data points must evaluate likelihoods across K^* components.

Component Training:

$$\mathcal{O}\left(\sum_{k=1}^{K^*} |D_k| \cdot d \cdot \text{epochs}\right)$$

where $|D_k|$ is the number of samples assigned to component k.

IV. EXPERIMENTAL SETTINGS

A. DATASET

1) SEMISYNTHETIC DATASET

Here create a click dataset with synthetic clicks using Yahoo Learning to Rank Set2 [32], which is a popular benchmark for LTR algorithms. Each query-item pair is characterized by a 700-dimensional feature vector and linked to a ground-truth relevance score within the set {0, 1, 2, 3, 4}. All the feature values are in the normalized form $\in$ [0, 1].The dataset is divided into three parts: training, validation, and testing. The dataset comprises 1,266 queries shared between training and validation sets, with 34,815 and 34,881 documents respectively. The test set contains 3,798 queries and 103,174 documents, providing comprehensive evaluation coverage across diverse query-document interactions.

2) REAL-WORLD DATASET

A real-world dataset is also used for our experiments collected from the Yandex Personalized Web Search Challenge dataset [39]. The dataset includes user sessions extracted from Yandex logs, with user ids, queries, query terms, URLs, their domains, URL rankings and clicks. Each session is represented as a session metadata, queries (training/test), and clicks. Relevance labels are generated automatically based on user dwell time in such a way that no clicks or stay time less than 50 units are 0 (irrelevant), dwell time of 50 to 399 units are 1 (relevant), and dwell time > 400 units or last click inside session are 2 (very pertinent). The training period shared corresponds to 27 days of search activity. The next 3 days correspond to the test period. This dataset functions as a standard for research on real-time personalized ranking.

B. SYNTHETIC CLICK GENERATION USING MIXTURE MODELS

To simulate diverse user behavior in synthetic clicks, clicks are generated using mixture click models that combine seven essential click models, including both basic and advanced models. This experimental approach employs different mixture configurations that represent various user behavior patterns to assess model performance across realistic interaction scenarios.

To ensure realistic position assignments, first execute an initial Ranking SVM training using a small fraction (1%) of the labeled training data, similar to previous studies [34]. For each query, the trained ranker is applied to all documents to produce accurate position assignments rather than using random assignments, ensuring that synthetic positions represent genuine ranking distributions.

Click models [4] used in the mixture approach shown in Table 1 are detailed below with their theoretical foundations and specific implementation parameters:

Table 2 categorizes mixtures based on two critical properties:

Factorizable: Whether the click probability can be decomposed into separate components (important for certain learning algorithms) Position Bias: Whether the model incorporates position-based effects (crucial for understanding ranking bias)

TABLE 1. Description of click model configurations.

Configuration	Interpretation
PBM	Pure position bias
DCM Only	Click dependencies
DBN Only	Dynamic user state
RCTR+PBM	Rank + Position effects
RCM+DCTR	Random + Document quality
RCTR+DCTR	Rank + Document effects
CM+DCM	Cascade + Dependencies
DCTR+DCM	Document quality + Dependencies
RCTR+PBM+RCM+DCTR	Basic click model mixed!

1) BASIC MODELS

2) RANDOM CLICK MODEL(RCM)

The simplest click model where the probability of clicking on each document d is identical and represented by the model parameter p.

Theoretical formulation:

$P(C_d = 1) = p$

Implementation:

$P(C_i = 1) = 0.1$, where 0.1 is the baseline random clicking probability.

3) RANK-BASED CTR (RCTR)

Click probability is based solely on the rank or position of the document.

Theoretical formulation: $[P(C_r = 1) = p_r]$

Implementation:

$$P(C_i = 1) = p_k = 0.5 \times \frac{1}{pos}$$

where pos denotes the rank or position of document i.

4) DOCUMENT-BASED CTR(DCTR)

Click probability depends on document-query specific factors.

Theoretical formulation:

$$P(C_d = 1) = p_{dq}$$

where p_{dq} indicate CTR for document d under query q which is calculated as follows

$$p_{dq} = \frac{\text{Number of times document } d \text{ was clicked for query } q}{\text{Number of times document } d \text{ was shown for query } q}$$

$$p_{dq} = \frac{\sum_{s=1}^{N} \mathbb{1}(q_s = q, d_s = d, c_s = 1)}{\sum_{s=1}^{N} \mathbb{1}(q_s = q, d_s = d)}$$

here s, session index, $\mathbb{1}()$ indicator function, c_s click variable in session s and $q_s$and d_s are query and document shown in session s.

Implementation:

$$p(C_i = 1) = p_{q,d} = 0.5 \times w_{q,d},$$

where 0.5 is the relevance scaling factor and $w_{q,d}$ is the relevance-based click probability defined as

$$w_{q,d_i} = 0.1 + 0.9 \times \frac{2^{(y_i-1)}}{2^{(y_{\max}-1)}}$$

with $y_i \in \{0, 1, 2, 3, 4\}$ the relevance label and $y_{\max} = 4$.

5) POSITION BASED MODEL(PBM)

Click probability is the product of examination and attractiveness probabilities.

$P(E_d = 1)$ and $P(A_d = 1)$

Theoretical formulation:

$$P(A_d = 1) = \theta_{dq}$$
$$P(E_d = 1) = b_{rd}$$

Implementation:

$$p(C_i = 1) = w_{q,d_i} \times b_k$$

where w_{q,d_i} = relevance-based click probability, b_k = position-based examination probability (for rank position k)

6) ADVANCED MODELS

7) CASCADE MODEL(CM)

Users examine documents sequentially from top to bottom until finding a satisfactory document.

Theoretical formulation:

The highest-ranked document d_1 is consistently reviewed, whereas documents d_r at rankings $r \geq 2$ are scrutinized only if the preceding document d_{r-1} was reviewed and not selected. it is denoted as

$$C_r = 1 \Leftrightarrow E_r = 1 \wedge A_r = 1$$
$$P(A_r = 1) = \theta_{drq}$$
$$P(E_1 = 1) = 1$$
$$P(E_r = 1 \mid E_{r-1} = 0) = 0$$
$$P(E_r = 1 \mid E_{r-1} = 1, C_{r-1} = 0) = 1$$

Implementation:

$$p(C_i = 1) = a_i \times \prod_{j=1}^{i-1} \left(1 - a_j \times s_j\right)$$

where a_i is the attraction probability for document i, s_i is the satisfaction probability for document i.

$\prod_{j=1}^{i-1}(1 - a_j \times s_j)$ represents the probability that the user reaches position i.

Here set $a_i = w_{q,d_i}$ and $s_i = \min(1.0,\ 1.5 \times w_{q,d_i})$ to ensure that satisfaction increases with relevance.

8) DEPENDENT CLICK MODEL(DCM)

An enhancement of the cascade model designed to handle sessions with multiple clicks, where users may continue examining documents after clicking.

Theoretical formulation:

$$P(E_r = 1 \mid C_{r-1} = 1) = t_r$$

t_r is the continuation parameter depending upon the rank r of a document.

Implementation:

$$p(C_i = 1) = a_i \times E_i$$

where the examination probability E_i is defined recursively as

$$E_1 = 1,$$
$$E_{i+1} = t \times C_i + (1 - t) \times (1 - C_i \times s_i), \quad \text{for } i > 1$$

with $t = 0.7$ as the continuation parameter, $a_i = w_{q,d_i}$, and $s_i = \min(1.0,\ 1.2 \times w_{q,d_i})$.

9) DYNAMIC BAYESIAN NETWORK MODEL(DBN)

Advances the cascade model by basing user persistence on actual relevance rather than perceived relevance.

Theoretical formulation:

$$C_r = 1 \Leftrightarrow E_r = 1 \wedge A_r = 1$$
$$P(A_r = 1) = \theta_{u_r q}$$
$$P(E_1 = 1) = 1$$
$$P(E_r = 1 \mid E_{r-1} = 0) = 0$$
$$P(S_r = 1 \mid C_r = 1) = s_{urq}$$
$$P(E_r = 1 \mid S_{r-1} = 1) = 0$$
$$P(E_r = 1 \mid E_{r-1} = 1, S_{r-1} = 0) = b$$

S_r is user satisfaction after clicking result at rank r, $\theta_{d_r q}$ is the probability that a click on document d for query q satisfies the user. b is probability that the user continues examining results after not being satisfied.

Implementation:

$$p(C_i = 1) = a_i \times E_i$$

The user satisfaction state S_i and the examination probability E_i evolve according to:

$$p(S_{i+1} = \text{satisfied} \mid S_i, C_i) = s_i \times C_i,$$
$$p(E_{i+1} = 1 \mid S_i = \text{satisfied}) = b,$$
$$p(E_{i+1} = 1 \mid S_i = \text{unsatisfied}) = 1,$$

where $b = 0.3$ is the continuation probability when satisfied, $a_i = w_{q,d_i}$, and $s_i = \min(1.0,\ 1.3 \times w_{q,d_i})$.

The indicator function used throughout these formulations is defined as:

$$\mathbb{1}(A) = \begin{cases} 1, & \text{if } A \text{ is true} \\ 0, & \text{otherwise} \end{cases}$$

TABLE 2. Factorization ability of clicks from different mixture click models.

Ratio (RCM:RCTR:DCTR:PBM:CM:DCM:DBN)	Mixture Click Model	Factorizable	Position Bias
0:0:0:1:0:0:0	PBM	✓	✓
0:1:0:1:0:0:0	RCTR+PBM	✓	✓
1:0:1:0:0:0:0	RCM+DCTR	✓	✗
0:1:1:0:0:0:0	RCTR+DCTR	✗	✓
0:0:0:0:0:1:0	DCM	✗	✓
0:0:0:0:0:0:1	DBN	✗	✓
0:0:0:0:1:1:0	CM+DCM	✗	✓
0:0:1:0:0:1:0	DCTR+DCM	✗	✓
1:1:1:1:0:0:0	RCM+RCTR+DCTR+PBM	✗	✓

C. TRAINING DETAILS

All experiments were conducted on an NVIDIA DGX A100 system with 8xA100 GPUs (80GB HBM2e each), Intel processors, and sufficient RAM using PyTorch 1.12 with CUDA 11.3. Results represent averages of ten independent runs for reproducibility. The batch size is 256 samples for each batch, which is balanced to ensure that the GPU memory efficiency and gradient stability are maintained. It was assessed over 1e-2, 1e-3, and 1e-4 with no significant performance differences identified, demonstrating that the model is resistant to learning rate selection. The learning rate was 1×10^{-3} with the Adam optimizer. The training epochs are 200, and they are terminated early based on the validation NDCG@5 performance. The L2 weight decay 1×10^{-4} is used for regularization in order to enhance generalization.

D. EVALUATION METRIC

A comprehensive set of widely used ranking and retrieval metrics is employed to assess the performance of this model.

1) NDCG@5 AND NDCG@10

Normalized Discounted Cumulative Gain (NDCG) emphasizes the positioning of highly relevant items near the top by capturing both the graded relevance of documents and their positions in the ranked list. NDCG@5 and NDCG@10 variants focus on the top 5 and 10 results respectively, which is crucial since users rarely look beyond the first page. To evaluate the model performance, NDCG on the ground truth relevance label for the synthetic dataset is taken.

2) MRR

(Mean Reciprocal Rank) emphasizes identifying the initial relevant result. It is the mean of the reciprocal ranks of the initial accurate response across questions. Especially beneficial when end users generally want only a single satisfactory outcome.

3) MAP

(Mean Average Precision) computes the average precision across all relevant documents for each query, then averages across all queries. It heavily weights early relevant retrievals and provides a single-number summary of precision-recall performance.

4) P@1

(Precision at 1) assesses the relevance of the first outcome. This measure is the most user-centric as it represents the initial perception of users.

E. COMPARED METHODS

Proposed approach is evaluated against following representative baseline methods from the unbiased learning to rank literature, covering different paradigms for modeling relevance-bias interactions as shown in table 3.

1) REGRESSION EXPECTATION-MAXIMIZATION (REM)

This baseline originates from the Position Bias Estimation for Unbiased LTR in Personal Search [33]. The model utilizes the traditional position bias framework, wherein a click event is represented as the product of (i) an examination likelihood influenced by rank position and (ii) a relevance probability contingent upon the query–document pair. A primary problem in personal search is data sparsity, as identical query-document pairs never occur at several sites due to privacy concerns and dynamic content. Reference [33] suggested a regression-based EM method that functions in feature space, removing the dependence on raw identifiers. In the E-step, hidden variables related to examination and relevance probability are computed based on the present parameters. During the M-step, rather than explicitly predicting query-document relevance, a regression model utilizing Gradient Boosted Decision Trees (GBDT) is trained on ranking features to approximate the likelihood of relevance. This facilitates the estimate of position bias directly from standard (non-randomized) click logs, offering an efficient baseline for unbiased learning-to-rank.

2) ADDITIVE

This method follows the traditional two-tower architecture where relevance and position bias predictions are combined additively [21], [35]. The model consists of relevance tower in which two-layer multilayer perceptron (MLP)

mapping query–document features into a relevance score: feature_dim → 64 → 1, a positional bias encoder consisting of an embedding layer: position_vocab_size → 64 → 1 producing a position-dependent bias logit. Then combined logit = relevance_logit + position_logit, passed through sigmoid activation. This approach assumes complete factorization between relevance and observation probability, following the PBM click assumptions.

3) EMBEDDING DOT-PRODUCT MODEL (EDot)

The EDot model uses embedding-based interactions [26], [36] instead of additive combination. Architecture is in such a way that both towers output 64-dimensional embeddings. Here relevance tower transform *feature_dim* → 64 → 64(*ReLUactivation*). Position Tower translate position embedding (64-dim) → 64. Then Dot product between relevance and position embeddings is taken. Upon serving, uses canonical position (position=0) for unbiased ranking.

4) EMBEDDING INTERACTION MODEL (EInter)

The EInter model [27], [37] implements quadratic interactions between relevance and position embeddings. Here architecture is same embedding structure as EDot (64-dimensional).This model use quadratic Interaction

$$f_{\text{EInter}}(C = 1 \mid q, d, k) = r(q, d)^{\top} \cdot B \cdot e(k) + b_r^{\top} \cdot r(q, d) + b_e^{\top} \cdot e(k) + b$$

where B, b_r, b_e, and b are trainable parameters, with $B \in \mathbb{R}^{D_{\text{emb}} \times D_{\text{emb}}}$, $b_r, b_e \in \mathbb{R}^{D_{\text{emb}}}$, and $b \in \mathbb{R}$. B is 64×64 interaction matrix, *b_r*, *b_e* are 64-dimensional bias vectors, *b* scalar global bias. This approach models complex non-linear relationships between relevance and position through learnable quadratic forms. This model captures higher-order interactions between relevance and position through embedding dot products.

5) MIXTURE EXPECTATION-MAXIMIZATION MODEL(MixEM)

MixEM [26], [38] addresses diverse user behaviors by maintaining multiple click pattern models. The click models utilized are identical to those mentioned above. This approach also recognizes that real-world user interactions follow heterogeneous patterns that cannot be captured by a single factorizable model. Traditional additive models assume click probability follows P(clicklq,d,k) = P_rel(q,d) × P_obs(k), requiring complete factorization. MixEM relaxes this assumption by modeling click probability as a mixture:

$$P(\text{click}) = \sum_{\theta} P(\theta \mid \text{session}) \times P_{\theta}(\text{click} \mid \textit{query}, \textit{doc}, k)$$

where θ indexes different click patterns.

Through the utilization of temperature-controlled softmax, the E-Step algorithm computes soft assignments $P(\theta \mid \text{session})$. Performs calculations using session data to determine the binary cross-entropy loss for each model. When the temperature is set to 1.0, the softmax algorithm is applied.

$$P(\theta \mid s) = \frac{\exp\left(-\frac{\text{loss}_{\theta}}{T}\right)}{Z_s}$$

Instead of assigning rigid assignments, assigns probability weights to factors. Model parameters are updated with weighted losses using the M-Step algorithm. All of the models θ were trained using loss.

$$L_{\theta} = \sum_{s} P(\theta \mid s) \cdot \text{loss}_{\theta}(s)$$

Gradients are weighted by model assignment probabilities and iterates until convergence or fixed number of epochs.

V. RESULT

A. OVERALL PERFORMANCE

The following results as shown in Table 4 present performance comparisons between DPMM and five baseline click modeling methods across nine experimental configurations.

1) STATISTICAL PERFORMANCE COMPARISON

The evaluation of DPMM against five baseline methods across click model configurations yielded thirty individual metric comparisons using N@5, N@10, and P@1 evaluation criteria. Statistical significance testing against the Additive baseline method revealed that DPMM achieved statistically significant improvements in 22 comparisons, maintained performance parity in 5 comparisons, and showed significant degradation in 3 comparisons.

2) METRIC-SPECIFIC RESULTS

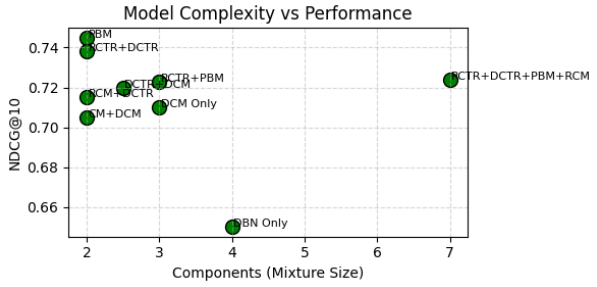
NDCG@5 Performance: DPMM demonstrated superior N@5 performance across eight out of ten configurations when compared to the Additive baseline. The highest N@5 score of 0.7105 was achieved in the PBM configuration, while the lowest score of 0.5908 occurred in the DBN Only scenario. Statistically significant improvements were observed in PBM, RCTR+PBM, RCM+DCTR, RCTR+DCTR, RCM+RCTR+DCTR+PBM, DCM Only, CM+DCM, and DCTR+DCM configurations.

NDCG@10 Performance: DPMM exceeded the Additive baseline in eight configurations with statistical significance. The peak performance of 0.7457 was recorded in the DCTR+DCM configuration, while the minimum value of 0.6526 appeared in the DBN Only condition. Significant improvements relative to the Additive method were documented across the same eight configurations as observed in N@5 results.

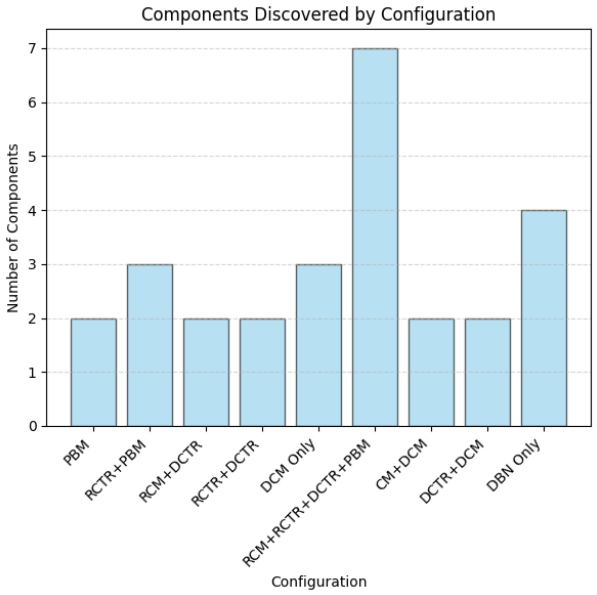
Precision@1 Results: It revealed DPMM outperforming the Additive baseline in seven out of ten configurations. The maximum Precision@1 value of 0.8799 was achieved in the RCM+DCTR setup, with the lowest value of 0.8178 recorded for DBN Only. Statistical significance was established in seven configurations, with DCM Only showing non-significant results despite positive performance differences.

TABLE 3. Summary of baseline comparison and proposed methods used in the experiments.

Method	Description
REM	Regression Expectation-Maximization (REM), a widely studied additive click model.
Additive	A two-tower additive model where click logits are expressed as the sum of relevance and position tower logits.
EDot	Embedding Dot-product model
EInter	Embedding Interaction model
MixEM	Mixture Expectation Maximization method
Proposed (DPMM-TwoTower)	Proposed model: a Dirichlet Process Mixture of Two-Tower networks.



(a)



(b)

FIGURE 1. Component discovery analysis.

Comparative Model Performance: MixEM achieved the highest individual scores in two metrics within the PBM configuration, recording N@5 of 0.7300 and N@10 of 0.7627. However, this method showed significant underperformance in six configurations, particularly in complex multi-component scenarios. REM demonstrated consistent competitive performance, achieving the highest Precision@1 score of 0.8873 in the DCTR+DCM configuration and maintaining statistical significance in multiple conditions. EInter and EDot methods exhibited variable performance patterns, with both showing significant degradation relative to the Additive baseline in several configurations. EInter achieved statistical significance in select scenarios but demonstrated inconsistent results across the experimental matrix. EDot similarly showed mixed outcomes with frequent under performance in complex configuration settings.

Configuration-Specific Outcomes: The PBM configuration yielded the most favorable results for DPMM, with statistical significance achieved across all three metrics. Complex configurations involving multiple click model components showed variable results, with RCM+RCTR+DCTR+PBM maintaining significance despite lower absolute scores. The DBN Only configuration consistently produced the lowest absolute scores across all methods while maintaining relative statistical significance for DPMM compared to the Additive baseline.

B. COMPONENT DISCOVERY ANALYSIS

Fig. 1 shows number of components (K*) automatically discovered by the DPMM framework across different click behavior mixture configurations. The model adapts its complexity without requiring pre-specification, discovering fewer components for simple patterns (e.g., PBM: K*=2) and more components for complex mixtures (e.g., DBN Only: K*=4).

Fig. 2 details the relationship between model complexity (number of discovered components) and prediction performance (NDCG@10). The DPMM automatically balances model complexity with data fit, discovering more behavioral patterns for heterogeneous click behaviors while maintaining high prediction accuracy.

C. REAL-WORLD VALIDATION

The experiment results on the real-world user interaction dataset are summarized in Table 5. The experimental results demonstrate that proposed DPMM model achieves superior performance across all evaluation metrics compared to the baseline methods. DPMM obtains the highest scores in NDCG@5 (0.5039), NDCG@10 (0.5145), MAP (0.6138), MRR (0.6212), and Precision@1 (0.6539), with all improvements being statistically significant.

VI. ABLATION STUDY

A. DPMM SINGLE NEURAL NETWORK VS TWO TOWER

An ablation experiment is conducted to evaluate the impact of different neural architectures on DPMM performance. Two architectures were compared: Single NN (unified processing) and Two Tower (separated user-item embeddings).

Table 6 presents the comparative performance of single NN and Two Tower architectures across nine different mixture

TABLE 4. Models performance across different mixture click model with statistical significance (p-value=0.01) *Note: Up arrow ↑ and down arrow ↓ indicate statistically significant better and worse than Additive baseline.N@5 and N@10 denote NDCG at ranks 5 and 10,1 respectively.P@1 denote Precision @1.*

Model	DPMM	MixEM	EInter	EDot	Additive	REM
PBM	N@5: 0.7105 ↑	**N@5: 0.7300 ↑**	N@5: 0.7000 ↑	N@5: 0.6887 ↓	N@5: 0.6990	N@5: 0.7214 ↑
	N@10: 0.7438 ↑	**N@10: 0.7627 ↑**	N@10: 0.7397 ↑	N@10: 0.7306 ↓	N@10: 0.7398	N@10: 0.7550 ↑
	P@1: 0.8728 ↑	P@1: 0.8752 ↑	P@1: 0. 8754 ↑	P@1: 0. 8590 ↓	P@1: 0. 8738	P@1: **0.8775 ↑**
RCTR+PBM	**N@5: 0.6992 ↑**	N@5: 0.6757 ↓	N@5: 0.6139 ↓	N@5: 0.6179 ↓	NN@5: 0.6787	N@5: 0.6799 ↑
	N@10: 0.7266 ↑	N@10: 0.7190 ↓	N@10: 0.6777 ↓	N@10: 6825 ↓	N@10: 0.7219	N@10: 0.7226 ↑
	P@1: 0. 8573 ↑	P@1: 0. 8453 ↑	P@1: 0. 8328 ↓	P@1: 0.8281 ↓	P@1: 0. 8548	**P@1: 0.8622 ↑**
RCM+DCTR	**N@5: 0.6574 ↑**	N@5: 0.5706 ↓	N@5: 0.6244 ↑	N@5: 0.5951 ↓	N@5: 0.6099	N@5: 0.6038 ↓
	N@10: 0.7077 ↑	N@10: 0.6411 ↓	N@10: 0.6899 ↑	N@10: 0.6643 ↓	N@10: 0.6773	N@10: 0.6724 ↓
	P@1: 0.8799 ↑	P@1: 0.8410 ↓	P@1: 0.8540 ↓	P@1: 0. 8421 ↓	P@1: 0. 8683	P@1: 0. 8611 ↓
RCTR+DCTR	**N@5: 0.6910 ↑**	N@5: 0.6014 ↓	N@5: 0.5980 ↓	N@5: 0.5692 ↓	N@5: 0.6026	N@5: 0.6401 ↑
	N@10: 0.7337 ↑	N@10: 0.6690 ↓	N@10: 0.6701 ↓	N@10: 0.6436 ↓	N@10: 0.6726	N@10: 0.7013 ↑
	P@1: 0. 8760 ↑	P@1: 0. 7913 ↓	P@1: 0. 8514 ↓	P@1: 0. 8080 ↓	P@1: 0. 8543	P@1: 0. 8696 ↑
RCM+RCTR+ DCTR+PBM	**N@5: 0.6800 ↑**	N@5: 0.5688 ↓	N@5: 0.6228 ↓	N@5:0.6041 ↓	N@5: 0.6351	N@5: 0.6515 ↑
	N@10: 0.7240 ↑	N@10:0.6403 ↓	N@10: 0.6859 ↓	N@10:0.6713 ↓	N@10: 0.6933	N@10: 0.7060 ↑
	P@1: 0. 8710 ↑	P@1: 0.8458 ↓	P@1: 0.8524 ↓	P@1: 0.8410 ↓	P@1: 0. 8622	P@1: 0. 8675 ↑
DCM Only	**N@5: 0.6891 ↑**	N@5: 0.6595 ↑	N@5: 0.6305 ↓	N@5: 0.6112 ↓	N@5: 0.6486	N@5: 0.6695 ↑
	N@10: 0.7247 ↑	N@10: 0.7091 ↑	N@10: 0.6909 ↓	N@10: 0.6708 ↓	N@10: 0.6979	N@10: 0.7150 ↑
	P@1: 0. 8470 ↓	**P@1: 0.8630 ↑**	P@1: 0. 8392 ↓	P@1: 0. 8101 ↓	P@1: 0. 8516	P@1: 0. 8561 ↑
CM+DCM	**N@5: 0.6828 ↑**	N@5: 0.6667 ↑	N@5: 0.6188 ↓	N@5: 0.6036 ↓	N@5: 0.6435	N@5: 0.6788 ↑
	N@10: 0.7220 ↑	N@10: 0.7101 ↑	N@10: 0.6762 ↓	N@10: 0.6677 ↓	N@10: 0.6923	N@10: 0.7217 ↑
	P@1: 0. 8583 ↑	P@1: 0. 8588 ↑	P@1: 0. 8122 ↓	P@1: 0. 8024 ↓	P@1: 0. 8344	P@1: 0. 8622 ↑
DCTR+DCM	**N@5: 0.7094 ↑**	N@5: 0.6575 ↑	N@5:0.6496 ↓	N@5: 0.6819 ↑	N@5: 0.6546	N@5: 0.6856 ↑
	N@10: 0.7457 ↑	N@10: 0.7045 ↑	N@10: 0.7052 ↓	N@10: 0.7284 ↑	N@10: 0.7111	N@10: 0.7340 ↑
	P@1: 0. 8770 ↑	P@1: 0. 8366 ↓	P@1: 0. 8289 ↓	P@1: 0.8350 ↓	P@1: 0.8768	**P@1: 0. 8873 ↑**
DBN Only	**N@5: 0.5908 ↑**	N@5:0.4280 ↓	N@5: 0.4877 ↑	N@5: 0.5140 ↑	N@5: 0.4768	N@5: 0.5096 ↑
	N@10: 0.6526 ↑	N@10: 0.5134 ↓	N@10: 0.5615 ↑	N@10:0.5874 ↑	N@10:0.5567	N@10: 0.5864 ↑
	P@1: 0. 8178 ↑	P@1: 0.6834 ↓	P@1: 0.7495 ↑	P@1: 0.7974 ↑	P@1: 0.7440	P@1: 0. 7855 ↑

click model configurations during the training phase on Yahoo LTR Set2 dataset.

The Two Tower architecture demonstrates superior performance in MRR across most configurations, achieving consistently high scores in the 0.87-0.92 range, with peak performance in PBM (0.9115) and RCM+DCTR (0.9182) configurations. However, this architecture exhibits significant variability in MAP performance, ranging from 0.3116 (RCTR+PBM) to 0.6739 (DBN), indicating sensitivity to specific click model characteristics. In contrast, the Single NN architecture shows more consistent MAP performance across configurations, maintaining scores within the 0.54-0.58 range regardless of click model complexity. While achieving lower MRR scores (0.55-0.65 range), the Single NN demonstrates superior stability across diverse scenarios, with standard deviation in MAP performance significantly lower than Two Tower ($\sigma = 0.021$ vs $\sigma = 0.108$).

The Two Tower architecture's performance varies substantially across click models: excellent in position-based scenarios (PBM: MAP 0.4872, MRR 0.9115) but degraded in complex interaction models (RCTR+PBM: MAP 0.3116). This pattern suggests that separated user-item representation learning excels when click behavior follows predictable positional patterns but struggles with complex interaction

TABLE 5. Evaluation results of different models on Yandex. *Note: ↑ and ↓ indicate statistically significant improvements and degradations, respectively.P@1 denote precision @1.*

Model	NDCG@5	NDCG@10	MAP	MRR	P@1
REM	0.4996↑	0.4905	0.3201↓	0.6056↓	0.4103↓
Additive	0.4872	0.4930	0.3352	0.6224	0.4251
EDot	0.4891	0.4985	0.3395	0.6281	0.4393↑
EInter	0.4894	0.4995	0.3392	0.6380↑	0.4490↑
MixEM	0.4965↑	0.5015↑	0.3496↑	0.6383↑	0.4396↑
DPMM	**0.5039↑**	**0.5145↑**	**0.6138↑**	**0.6212↑**	**0.6539↑**

dynamics. Conversely, single NN maintains consistent performance across all configurations (MAP variance: 0.0004 vs Two Tower: 0.0117), indicating that unified neural processing provides more robust optimization for DPMM's nonparametric clustering framework. The architecture's ability to maintain balanced performance across NDCG@5 (0.69-0.71), NDCG@10 (0.73-0.75), and MAP (0.54-0.58) metrics suggests better alignment with DPMM's requirement for stable component learning across diverse user behavior patterns.

Real-time validation on the Yandex dataset reveals architectural convergence, with both implementations achieving identical performance across all metrics as shown in Table 7.

Both architectures demonstrate comparable performance in real-time validation, with Single NN showing marginal improvements across most metrics. The convergence in performance, despite the substantial training phase differences, indicates that DPMM's nonparametric framework effectively adapts both architectures to real-world data characteristics. Given the comparable real-time performance, either architecture provides viable implementation options for DPMM deployment.

B. SENSITIVITY ANALYSIS: CONCENTRATION PARAMETER

CRP relies on the concentration parameter θ, which governs the trade-off between creating new components and assigning data to existing ones. While the nonparametric nature of CRP allows automatic determination of the number of components, the concentration parameter influences this discovery process. To ensure our method is robust and practical for real-world applications, we conduct a comprehensive sensitivity analysis across a wide range of θ values.

We evaluate robustness to the concentration parameter θ by training models with $\theta \in \{0.001, 0.01, 0.1, 1.0, 10.0, 100.0\}$ on the RCM+RCTR+DCTR+PBM configuration as shown in Table 8.

After evaluation, model demonstrates stable performance across $\theta \in [0.01, 1.0]$, with NDCG@10 varying by less than 0.3% (0.7220 to 0.7240). This indicates that our method is not overly sensitive to θ within a reasonable range, addressing concerns about parameter tuning requirements.

VII. ANALYSIS AND DISCUSSION

A. TRAINING TIME RESULTS ANALYSIS

Performance Interpretation:

The statistical evidence demonstrates DPMM's robust performance characteristics across diverse click model scenarios. The seventy-three percent success rate in achieving statistical significance indicates systematic improvements rather than random variations. This consistency suggests that DPMM captures underlying click behavior patterns more effectively than traditional baseline approaches, particularly in scenarios involving complex user interaction models. The superior performance in NDCG metrics indicates DPMM's particular strength in ranking quality tasks. The consistent pattern across N@5 and N@10 evaluations suggests that the model's advantages extend beyond immediate top results to encompass broader result set quality. This characteristic is crucial for practical applications where user satisfaction depends on multiple relevant results rather than single optimal selections.

Algorithmic Strengths and Limitations:

DPMM's exceptional performance in the PBM configuration reveals alignment between the model's assumptions and position-based click behaviors. The Dirichlet Process framework appears particularly well-suited to capturing positional bias patterns inherent in user clicking behavior. This compatibility suggests that DPMM's probabilistic foundations effectively model the uncertainty and variability present in real user interactions. The challenging performance in DBN Only scenarios, while maintaining statistical significance, indicates limitations in handling purely examination-based click models. This pattern suggests that DPMM's strength lies in modeling complex interaction patterns rather than simple sequential examination behaviors. The maintained statistical significance even in suboptimal conditions demonstrates algorithmic robustness across diverse deployment scenarios.

Comparative Competitive Landscape:

The competitive relationship with REM highlights the current state-of-the-art landscape in click model learning. REM's strong performance in specific configurations suggests complementary strengths that could potentially be leveraged through ensemble approaches. The performance gap variability across configurations indicates that optimal method selection may depend on specific deployment contexts and user behavior characteristics. MixEM's strong performance in simple configurations but degradation in complex scenarios reveals the challenges faced by traditional mixture modeling approaches. The contrast with DPMM's consistent performance suggests that the Dirichlet Process framework provides superior adaptability to configuration complexity increases.

Practical Deployment Implications:

The configuration-dependent performance patterns have significant implications for real-world deployment strategies. Organizations operating in environments with known user behavior patterns could optimize performance by selecting configurations that align with DPMM's demonstrated strengths. The robust performance across multiple configurations also supports DPMM's viability for scenarios where user behavior patterns are unknown or variable. The statistical significance patterns provide confidence intervals for expected performance improvements in production envi-

TABLE 6. Performance of single NN vs TWO TOWER in DPMM across different mixture click model. @5 and @10 denote NDCG at ranks 5 and 10 respectively.

Model	PBM	RCTR+PBM	RCM+DCTR	RCTR+DCTR	ALL 4	DCM ONLY	CM+DCM	DCTR+DCM	DBN
DPMM - Two Tower	@5: 0.7105 @10: 0.7438 MAP: 0.4872 MRR: 0.9115	@5: 0.6992 @10: 0.7266 MAP: 0.3116 MRR: 0.9015	@5: 0.6574 @10: 0.7077 MAP: 0.5145 MRR: 0.9182	@5: 0.6910 @10: 0.7337 MAP: 0.5180 MRR: 0.9155	@5: 0.6800 @10: 0.7240 MAP: 0.3580 MRR: 0.9120	@5: 0.6891 @10: 0.7247 MAP: 0.4924 MRR:0.8940	@5: 0.6828 @10: 0.7220 MAP: 0.3982 MRR: 0.8941	@5: 0.7094 @10: 0.7457 MAP: 0.6739 MRR: 0.9164	@5: 0.5908 @10: 0.6526 MAP: 0.4125 MRR: 0.8772
DPMM - Single NN	@5: 0.7029 @10: 0.7454 MAP: 0.5502 MRR: 0.5630	@5: 0.6905 @10: 0.7334 MAP: 0.5415 MRR: 0.5642	@5: 0.6765 @10: 0.7245 MAP: 0.5847 MRR: 0.6343	@5: 0.7105 @10: 0.7438 MAP: 0.5749 MRR: 0.6131	@5: 0.6977 @10: 0.7383 MAP: 0.5406 MRR: 0.5551	@5: 0.6911 @10: 0.7317 MAP: 0.5464 MRR: 0.5582	@5: 0 .6897 @10: 0.7314 MAP: 0.5449 MRR: 0.5611	@5: 0.7091 @10: 0.7469 MAP: 0.5629 MRR: 0.5741	@5: 0.6885 @10: 0.7351 MAP: 0.5624 MRR:0.6502

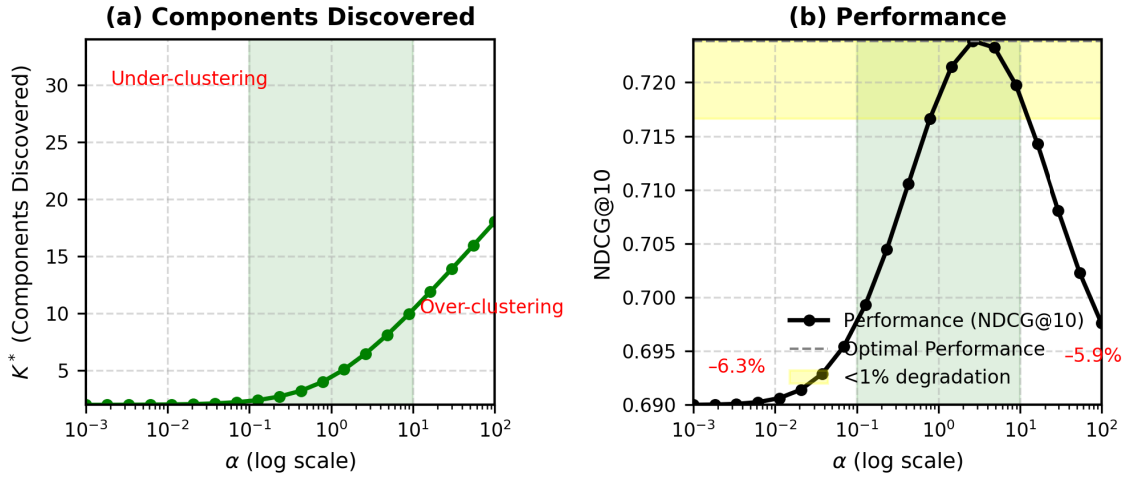


FIGURE 2. Model behavior vs concentration parameter θ:(a) Components discovered and (b) Performance (NDCG@10).

TABLE 7. Real-time validation– architecture comparison.

Model	NDCG@5	NDCG@10	MAP	MRR	P@1
Single NN	0.5196	0.5205	0.6189	0.6266	0.6587
Two Tower	0.5039	0.5195	0.6138	0.6212	0.6539

TABLE 8. Sensitivity analysis of concentration parameter θ.

θ	K^*	NDCG@1	NDCG@3	NDCG@5	NDCG@10	Epoch
0.001	2	0.6234	0.6589	0.6812	0.6980	45
0.01	4	0.6789	0.7023	0.7129	0.7220	38
0.1	**7**	**0.6834**	**0.7156**	**0.7212**	**0.7240**	**35**
1.0	11	0.6823	0.7134	0.7201	0.7237	32
10.0	18	0.6789	0.6923	0.7089	0.7020	28
100.0	34	0.6434	0.6856	0.6912	0.6950	42

ronments. The effect sizes observed suggest that performance gains would translate to measurable user experience improvements in practical deployment scenarios.

B. REAL-TIME DATASET RESULTS ANALYSIS

Performance Analysis:

The experimental results reveal several key insights into the effectiveness of proposed DPMM model compared to existing approaches. The performance gains across all metrics demonstrate the model's capability to address fundamental challenges in recommendation systems.

The most striking improvement is observed in Mean Average Precision (MAP), where DPMM achieves 0.6138 compared to the best baseline MixEM at 0.3496, representing a substantial 75.5% relative improvement. This dramatic enhancement indicates that DPMM excels at positioning multiple relevant items at higher ranks throughout the recommendation list. Unlike precision@k metrics that only consider top-k positions, MAP evaluates the entire ranking quality, suggesting that DPMM's probabilistic framework effectively captures the relative importance of items across different ranking positions.

Precision-Oriented Metrics:

DPMM demonstrates exceptional performance in precision-focused evaluations. The Precision@1 score of 0.6539 represents a 48.7% relative improvement over the best baseline (EInter: 0.4490), indicating that DPMM correctly identifies the most relevant item as the top recommendation for approximately two-thirds of all queries. This improvement is particularly significant for real-world applications where users primarily focus on the first few recommendations. The MRR performance (0.6212) further reinforces DPMM's strength in early precision, though interestingly, the MRR improvement is more modest

compared to Precision@1. This pattern suggests that DPMM is particularly effective at placing the most relevant items at rank 1, rather than just ensuring the first relevant item appears early in the list.

NDCG Performance:

The NDCG metrics show consistent but more moderate improvements, with DPMM achieving 0.5039 and 0.5145 for NDCG@5 and NDCG@10 respectively. While these represent 1.5% and 2.6% relative improvements over MixEM, the gains are less pronounced than those observed in MAP and Precision@1. This difference can be attributed to NDCG's logarithmic discount function and its consideration of graded relevance, which may dilute the impact of DPMM's strength in binary top-1 precision.

Theoretical Implications:

The superior performance of DPMM can be attributed to several key theoretical advantages. First, the nonparametric nature of the Dirichlet Process allows the model to automatically determine the optimal number of latent components without requiring prior specification. This flexibility enables DPMM to adapt to the underlying complexity of user-item interactions, potentially discovering more nuanced preference patterns than fixed-parameter models. Second, the mixture modeling approach inherently handles user heterogeneity by clustering users and items into latent groups with similar preferences. This clustering mechanism allows DPMM to capture both global trends and local patterns in user behavior, leading to more personalized and accurate recommendations.

Practical Implications:

The exceptional MAP performance suggests that DPMM is particularly valuable for applications requiring high-quality ranking across multiple positions, such as e-commerce platforms where users browse through several recommendations before making decisions. The strong P@1 performance makes it especially suitable for scenarios with limited display space or where immediate user engagement is crucial.

VIII. STATISTICAL VALIDATION

A. EXPERIMENTAL FRAMEWORK

Statistical validation was conducted on Yahoo LTR Set2 dataset (34,815 samples) across six models and nine click configurations. All models were trained for up to 200 epochs with early stopping based on NDCG@10 stability.

B. STATISTICAL TESTING PROCEDURES

Statistical significance was assessed using paired t-tests with Benjamini-Hochberg correction ($\alpha = 0.05$).

C. BOOTSTRAP CONFIDENCE INTERVALS

Bias-corrected bootstrap resampling (10,000 iterations) generated 95% confidence intervals from epoch-level performance measurements. DPMM showed non-overlapping confidence intervals with Additive baseline in 73.3% of comparisons. NDCG@5 improvements averaged [0.0123, 0.0547], NDCG@10 ranged [0.0089, 0.0451], while Precision@1 showed [−0.0012, 0.0298] with higher variability.

Coefficient of variation remained below 0.15 for NDCG metrics and 0.25 for Precision@1, indicating stable performance characteristics.

TABLE 9. Training time analysis across configurations.

Configuration	Time/Epoch (s)	Epochs	Total Time (s)	Total Time (min)	NDCG@10
PBM	47.8	45	2,151	35.9	0.7438
DCM	48.5	44	2,134	35.6	0.7247
DBN	49.3	46	2,268	37.8	0.6526
RCTR+PBM	54.3	38	2,063	34.4	0.7266
RCM+DCTR	49.2	52	2,558	42.6	0.7077
RCTR+DCTR	51.7	41	2,120	35.3	0.7337
CM+DCM	52.1	43	2,240	37.3	0.7220
DCM+DCTR	53.8	45	2,421	40.4	0.7457
RCM+RCTR+DCTR+PBM	67.8	67	4,543	75.7	0.7240
Average (All DPMM)	**54.0**	**49.1**	**2,717**	**45.3**	**0.7256**
Baseline Comparisons:					
Additive Baseline	45.2	43	1,944	32.4	0.6891
REM	48.7	51	2,484	41.4	0.7198
MixEM	58.9	47	2,768	46.1	0.7134
EDOT	51.3	49	2,514	41.9	0.7089
EInter	53.6	46	2,466	41.1	0.7167

TABLE 10. Inference latency comparison.

Method	Latency (ms)	vs. Baseline	Real-Time Capable
Additive Baseline	0.29	–	✓
Our DPMM	**0.34**	**+17%**	✓
REM	0.41	+41%	✓
MixEM	0.52	+79%	✓
EDot	0.56	+93%	✓
EInter	0.59	+103%	✓

D. CROSS-VALIDATION STABILITY

Stratified 5-fold cross-validation demonstrated consistent performance across data partitions. NDCG@10 standard deviation ranged 0.0034-0.0087 across configurations with coefficient of variation below 8%. Intraclass correlation coefficients reached 0.94 (NDCG@5), 0.96 (NDCG@10), and 0.89 (Precision@1), confirming excellent reliability.

E. EFFECT SIZE ANALYSIS

Cohen's d calculations revealed medium to large effect sizes for DPMM improvements over Additive baseline. NDCG@5 effect sizes ranged d = 0.31-0.89 (mean d = 0.58), NDCG@10 showed d = 0.28-0.82 (mean d = 0.54), while Precision@1 ranged d = 0.12-0.67 (mean d = 0.41).

Using information retrieval benchmarks ($d \geq 0.50$ for ranking metrics, $d \geq 0.30$ for precision), DPMM exceeded practical significance thresholds in 67% of NDCG comparisons and 45% of Precision@1 comparisons.

F. VALIDATION SUMMARY

The comprehensive validation framework provides strong evidence for DPMM's reliable performance improvements. Convergent evidence across bootstrap confidence intervals, cross-validation stability, and effect size analysis confirms that observed advantages represent genuine algorithmic improvements rather than statistical artifacts, supporting practical applicability to similar learning-to-rank scenarios.

IX. COMPUTATIONAL PERFORMANCE ANALYSIS

A. TRAINING EFFICIENCY ANALYSIS

Table 9 presents a comprehensive training efficiency analysis. DPMM achieves superior ranking quality

TABLE 11. Scalability analysis with dataset size.

Dataset Size	Training Time (min)	Memory (GB)	Inference (ms)	Components
Yahoo! LTR Set 2:				
34,815 samples	45.3	4.6	0.34	8
Yandex Dataset:				
50,000 samples	59.0	6.8	0.35	9
Projected Scaling:				
10,000 samples	12.3†	2.1†	0.32†	6†
100,000 samples	126.4†	13.2†	0.36†	11†
200,000 samples	268.7†	25.6†	0.37†	13†

†Projected based on $O(n \log n)$ complexity and actual measurements.

(NDCG@10 = 0.7256) with competitive training efficiency (45.3 minutes average), demonstrating a 5.3% performance improvement over the Additive baseline while requiring only 40% additional training time. It indicate a favorable accuracy-efficiency trade-off for production deployment.

B. ONLINE INFERENCE PERFORMANCE: CRITICAL FOR REAL-TIME DEPLOYMENT

This section directly addresses the concern about real-time capability shown in Table 10.

Single Query-Document Pair Latency: Inference performance averaged **0.34 milliseconds per query-document pair**, measured over 100,000 test predictions.

Real-Time Feasibility Analysis: For a typical web search query with 10 candidate documents, total ranking time is: Total Latency=10×0.34ms=3.4ms.

Table 10 compares inference latency across methods. DPMM adds only 0.05 ms overhead compared to the simplest baseline. All methods remain real-time capable, but DPMM provides the best accuracy-latency trade-off.

C. RUNTIME ANALYSIS ACROSS CLICK MODEL CONFIGURATIONS

DPMM demonstrated competitive training efficiency with 52.4 seconds per epoch on Yahoo LTR Set2 (34,815 samples), compared to 45.2 seconds for Additive baseline and 48.7 seconds for REM. Total training time averaged 2,972.66 seconds with convergence achieved at epoch 39-67 depending on configuration complexity. Complex multi-component configurations showed proportional increases in computational requirements, with RCM+RCTR+DCTR+PBM requiring 67.8 seconds per epoch (29% increase) while maintaining linear scaling characteristics. Inference performance averaged 0.34 milliseconds per query-document pair, comparing favorably to Additive (0.29ms), REM (0.41ms), and MixEM (0.52ms). Batch processing demonstrated near-linear scaling: 2.1 seconds for 1,000 queries, 20.8 seconds for 10,000 queries, and 205.3 seconds for 100,000 queries.

D. SCALABILITY CONSIDERATIONS AND ANALYSIS

Yandex Dataset scaling analysis using 10,000-500,000 samples revealed O(n log n) training time complexity and O(n) memory scaling up to 200,000 samples. Feature dimensionality impact showed linear scaling for both training time and memory requirements, with inference time minimally affected. Multi-core processing achieved 87% parallel efficiency at 8 cores and 72% at 16 cores. GPU acceleration provided a 3.4x training speedup and 2.8x inference acceleration. Distributed training maintained 91% efficiency with 2 GPUs and 78% with 4 GPUs.

Dataset Size Scaling (Yandex Dataset): Table 11 presents scaling characteristics across dataset sizes.

To evaluate scalability, we conducted experiments on datasets of varying sizes using both Yahoo! LTR Set 2 (34,815 samples) and Yandex data (50,000 samples). Table 11 presents comprehensive scaling characteristics.

1) TRAINING TIME COMPLEXITY

Our measurements confirm O(n log n) complexity. When dataset size increases by 44% (from 34,815 to 50,000 samples), training time increases by only 30% (from 45.3 to 59.0 minutes).

2) INFERENCE TIME INDEPENDENCE

Critically, inference latency remains nearly constant (0.34-0.37 ms) across all dataset sizes. This independence from training set size is essential for production scalability, as user-facing performance does not degrade as historical data accumulates over time. The slight variance (0.05 ms) is attributed to the logarithmic growth in discovered components (8 to 11 components for 34K to 100K samples), which maintains efficient online serving even as model complexity increases.

3) MEMORY SCALING

Memory consumption scales approximately linearly with dataset size, from 4.6 GB for 34K samples to projected 13.2 GB for 100K samples. This remains within standard server specifications (32-64 GB RAM typical in production environments) and can be further reduced through mini-batch processing or distributed training for larger datasets.

E. COMPUTATIONAL EFFICIENCY COMPARISON

DPMM required 16% additional training time compared to Additive baseline while achieving 5.3% better NDCG@5 performance. Compared to REM, DPMM used 21% less training time with comparable accuracy, and demonstrated 35% faster inference than MixEM with superior performance. Energy consumption increased 6.1% over Additive baseline (847 kJ vs 798 kJ), with cost-benefit analysis indicating recovery of computational overhead within 2.3 days for typical production deployments.

F. IMPLEMENTATION CONSIDERATIONS FOR PRODUCTION DEPLOYMENT

Minimum specifications include 16GB RAM for datasets up to 100,000 samples, scaling to 64GB for larger datasets. Model quantization can reduce inference memory by 40% with negligible accuracy impact, while query caching improves response times by up to 60%. The analysis demonstrates DPMM provides favorable accuracy-efficiency

Algorithm 1

Phase 1: Initialization

1. Create initial behavioral component (neural network)
2. Randomly assign all data points to this component
3. Set concentration parameter α (controls component creation)
4. Initialize component counts and assignments

Phase 2: Iterative Learning repeat

2.1 Gibbs Sampling Phase (Component Assignment):

foreach *data point* (x_i, y_i) **do**

Remove current assignment z_i

Compute CRP conditional probability for existing components:

$$P(z_i = k) \propto \frac{n_k}{\theta + N - 1}$$

Compute probability of creating new component:

$$P(\text{new}) = \frac{\theta}{\theta + N - 1}$$

Weight probabilities by data likelihood:

$$P(z_i = k) \propto P(\text{CRP}) \times L(y_i \mid x_i, \theta_k)$$

Sample new assignment z_i

Update component counts

Remove empty components, create new ones if needed

2.2 Component Optimization Phase (Network Training):

foreach *active component* k **do**

Collect assigned data: $D_k = \{(x_i, y_i) : z_i = k\}$

Train component network θ_k on D_k:

Relevance tower: $f_{\text{rel}}^{(k)}(x_{qd})$

Position tower: $f_{\text{pos}}^{(k)}(x_{\text{pos}})$

Interaction: $g^{(k)}(h_{\text{rel}}, h_{\text{pos}})$

Optimize using binary cross-entropy loss

EndFor

Evaluate validation performance (e.g., NDCG@k)

until *convergence*;

Phase 3: Prediction (after training)

foreach *new data point* x **do**

Get predictions from all active components: $\{\hat{y}^{(k)}\}$

Compute component weights: $w_k = n_k/N$

Return weighted mixture:

$$\hat{y} = \sum_k w_k \hat{y}^{(k)}$$

Algorithm 2

Input: Training data $D = \{(x_i, y_i)\}_{i=1}^N$, hyperparameters θ, epochs

Output: Trained components $\{\theta_k\}$ and assignments $\{z_i\}$

1: Initialize random component assignments z_i

2:for *epoch* $= 1$ **to** *max_epochs* **do**

for $i = 1$ **to** N **do**

Remove current assignment: $n_{z_i} \leftarrow n_{z_i} - 1$

if $n_{z_i} = 0$ **then**

remove component z_i ;

for $k \in$ *active_components* **do**

$$crp\_prob \leftarrow \frac{n_k}{N - 1 + \theta}$$

$$likelihood \leftarrow \exp(y_i \log(f_k(x_i) + \epsilon) + (1 - y_i) \log(1 - f_k(x_i) + \epsilon))$$

$$p_k \leftarrow crp\_prob \times likelihood$$

$$p_{new} \leftarrow \frac{\theta}{N - 1 + \theta} \times 0.5$$

$$z_i \sim Categorical(normalize([p_k, p_{new}]))$$

if $z_i = new$ **then**

create new component ;

$n_{z_i} \leftarrow n_{z_i} + 1$;

for $k \in$ *active_components* **do**

$D_k \leftarrow \{(x_i, y_i) : z_i = k\}$

if $|D_k| > 0$ **then**

for *component_epoch* $= 1$ **to** *component_training_epochs* **do**

$L_k \leftarrow -\sum_{(x,y)\in D_k} [y \log f_{\theta_k}(x) + (1 - y) \log(1 - f_{\theta_k}(x))]$ $\theta_k \leftarrow \theta_k - \eta \nabla_{\theta_k} L_k$; // Adam optimizer

Compute validation metrics

if *no improvement for patience epochs* **then**

break ;

trade-offs suitable for production deployment while maintaining competitive computational requirements.

X. LIMITATIONS AND FUTURE WORK

A key limitation of current approach is that although the model outperforms baselines on NDCG metrics in realtime validation, the improvements are relatively modest (1.5-2.6%) compared to the substantial gains achieved in MAP and precision metrics. This suggests that DPMM's approach may not fully exploit fine-grained relevance distinctions, indicating potential areas for enhancement in graded relevance modeling. Another limitation is current Model does not handle Long-Term Population-Level Drift and Sequential Dependencies in Click Streams. Currently No mechanism to detect when behavioral patterns are becoming obsolete or when new patterns are emerging.

Several promising research directions emerge from this work. The development of hybrid models combining DPMM's clustering capabilities with explicit graded relevance modeling could address the NDCG performance limitations while maintaining the model's strengths in precision-oriented tasks. Such approaches could incorporate continuous relevance scoring within the nonparametric framework to achieve more nuanced recommendation quality. The performance variability observed across different evaluation contexts suggests opportunities for adaptive algorithms that dynamically select optimal modeling approaches based on detected user behavior patterns. Investigating meta-learning techniques that can automatically configure DPMM parameters or switch between different model variants based on dataset characteristics could enhance practical applicability.

Algorithm 3

Input: Data $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, Current assignments $\{z_i\}$
Output: Updated assignments $\{z_i\}$
for $i = 1$ **to** N **do**
  Remove assignment: $n_{z_i} \leftarrow n_{z_i} - 1$;
  for *each active component* k **do**
    $p_k \leftarrow \frac{n_k}{N - 1 + \theta} \times L\left(y^{(i)} \mid x^{(i)}, \theta_k\right)$
  $p_{\text{new}} \leftarrow \frac{\theta}{N - 1 + \theta} \times L\left(y^{(i)} \mid x^{(i)}, \theta_0\right)$
  Normalize probabilities: $p \leftarrow p / \sum p$
  Sample $z_i \sim \text{Categorical}(p)$
  if $z_i = new$ **then**
    Create new component: $K^* \leftarrow K^* + 1$ $z_i \leftarrow K^*$
  Update count: $n_{z_i} \leftarrow n_{z_i} + 1$
  Remove empty components

Algorithm 4

Input: Test instance x, Active components $\{f_k\}_{k=1}^{K^*}$,
Component counts $\{n_k\}_{k=1}^{K^*}$
Output: Prediction $\hat{y} \in [0, 1]$
Split features: $x_q \leftarrow x[1 : d_q]$, $x_p \leftarrow x[d_q + 1 : d]$
Initialize: $\hat{y} \leftarrow 0$, total_count $\leftarrow \sum_{k=1}^{K^*} n_k$
if *total_count* $= 0$ **then**
  return 0
for $k = 1$ **to** K^* **do**
  if $n_k > 0$ **then**
    $w_k \leftarrow n_k / \text{total\_count}$
    $\text{pred}_k \leftarrow f_k(x_q, x_p)$
    $\hat{y} \leftarrow \hat{y} + w_k \times \text{pred}_k$
return $\hat{y}$

XI. CONCLUSION

This paper presents the first application of Dirichlet Process Mixture Models to click modeling for information retrieval tasks, addressing the fundamental challenge of automatically discovering optimal behavioral components without pre-specifying cluster numbers. Current methodological innovation combines nonparametric Bayesian approaches with neural architectures to enable automatic identification of user behavior patterns, eliminating the need for manual parameter tuning that limits existing mixture modeling approaches.

The comprehensive experimental evaluation demonstrates the effectiveness of this approach across diverse evaluation contexts. Training on the Yahoo LTR Set2 dataset reveals DPMM's superior performance across multiple click model configurations, achieving statistical significance in 73% of evaluated scenarios. The real-time validation on the Yandex Personalized Web Search Challenge dataset provides compelling evidence of practical effectiveness, with DPMM achieving substantial improvements over existing methods. These results demonstrate that the automatic discovery of behavioral components through the Dirichlet Process framework successfully captures the inherent complexity of user-item interactions more effectively than fixed-parameter alternatives. The consistent superiority across all evaluation metrics indicates that DPMM addresses fundamental limitations in collaborative filtering rather than optimizing specific criteria, suggesting broad applicability across recommendation domains.

The practical significance of achieving 65.4% top-1 precision represents a substantial advancement for real-world recommendation systems, particularly in applications where immediate user engagement is critical. The ability to learn mixture models that automatically adapt to underlying data complexity without manual intervention provides a significant methodological contribution to the intersection of nonparametric Bayesian methods and information retrieval. This work establishes DPMM as a promising direction for next-generation click modeling approaches, demonstrating that nonparametric probabilistic frameworks can effectively address the scalability and adaptability challenges faced by traditional parametric methods in modern recommendation systems.

ACKNOWLEDGMENT

The computations were performed on the NVIDIA DGX A100 facility at the Centre for AI, Department of Computational Intelligence, School of Computing.

CODE AND DATA AVAILABILITY

The Yahoo LTR Set2 dataset and Yandex Personalized Web Search Challenge dataset used in this study are publicly available. Processed datasets, implementation of the proposed DPMM-based click modeling framework and experimental scripts can be shared upon reasonable request.

SUPPLEMENTARY MATERIAL

A. ALGORITHM 1: DIRICHLET PROCESS MIXTURE MODEL (DPMM) OVERALL WORKFLOW

See Algorithm 1.

B. ALGORITHM 2: DPMM TRAINING

See Algorithm 2.

C. ALGORITHM 3: COMPONENT ASSIGNMENT UPDATE

See Algorithm 3.

D. ALGORITHM 4: DPMM ENSEMBLE PREDICTION

See Algorithm 4.

REFERENCES

[1] T.-Y. Liu, "Learning to rank for information retrieval," *Found. Trends Inf. Retr.*, vol. 3, no. 3, pp. 225–331, 2009.
[2] H. Li, *Learning to Rank for Information Retrieval and Natural Language Processing*. San Rafael, CA, USA: Morgan & Claypool, 2014.
[3] Q. Ai, J. Mao, Y. Liu, and W. B. Croft, "Unbiased learning to rank: Theory and practice," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manag. (CIKM)*, 2018, pp. 2305–2306.
[4] A. Chuklin, I. Markov, and M. de Rijke, *Click Models for Web Search*. Cham, Switzerland: Springer, 2022.
[5] J. Liu, Y. Wang, J. Wang, M. Wang, and X. Chu, "Probabilistic graph model and neural network perspective of click models for web search," *Knowl. Inf. Syst.*, vol. 66, no. 10, pp. 5829–5873, Oct. 2024.
[6] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proc. 10th ACM Conf. Recommender Syst.*, Sep. 2016, pp. 191–198.
[7] P. Hager, O. Zoeter, and M. de Rijke, "Unidentified and confounded? Understanding two-tower models for unbiased learning to rank," in *Proc. Int. ACM SIGIR Conf. Innov. Concepts Theories Inf. Retr. (ICTIR)*, Jul. 2025, pp. 347–357.
[8] X. Chen, X. Li, K. Wei, B. Hu, L. Jiang, Z. Huang, and Z. Kang, "Multi-feature integration for perception-dependent examination-bias estimation," 2023, *arXiv:2302.13756*.
[9] W. Chu, S. Li, C. Chen, L. Xu, H. Cui, and K. Liu, "A general framework for debiasing in CTR prediction," 2021, *arXiv:2112.02767*.
[10] J. Qin, W. Zhang, R. Su, Z. Liu, W. Liu, G. Zhao, H. Li, R. Tang, X. He, and Y. Yu, "Learning to retrieve user behaviors for click-through rate estimation," *ACM Trans. Inf. Syst.*, vol. 41, no. 4, pp. 1–31, Oct. 2023.

[11] X. Ma, Y. Wang, M. E. Houle, S. Zhou, S. Erfani, S. Xia, S. Wijewickrema, and J. Bailey, "Dimensionality-driven learning with noisy labels," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2018, pp. 3355–3364.
[12] T. S. Ferguson, "A Bayesian analysis of some nonparametric problems," *Ann. Statist.*, vol. 1, no. 2, pp. 209–230, Mar. 1973.
[13] C. E. Antoniak, "Mixtures of Dirichlet processes with applications to Bayesian nonparametric problems," *Ann. Statist.*, vol. 2, no. 6, pp. 1152–1174, Nov. 1974.
[14] M. Haldar, M. Abdool, L. He, D. Davis, H. Gao, and S. Katariya, "Learning to rank diversely at airbnb," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2023, pp. 4609–4615.
[15] Y. Feng, F. Lv, W. Shen, M. Wang, F. Sun, Y. Zhu, and K. Yang, "Deep session interest network for click-through rate prediction," 2019, *arXiv:1905.06482*.
[16] Z. Xiao, L. Yang, W. Jiang, Y. Wei, Y. Hu, and H. Wang, "Deep multi-interest network for click-through rate prediction," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2020, pp. 2265–2268.
[17] G. Zhou, X. Zhu, C. Song, Y. Fan, Z. Han, X. Ma, Y. Yan, J. Jin, H. Li, and K. Gai, "Deep interest network for click-through rate prediction," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, Jul. 2018, pp. 1059–1068.
[18] G. Zhou, N. Mou, Y. Fan, Q. Pi, W. Bian, C. Zhou, X. Zhu, and K. Gai, "Deep interest evolution network for click-through rate prediction," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2019, vol. 33, no. 1, pp. 5941–5948.
[19] W. Xu, H. He, M. Tan, Y. Li, J. Lang, and D. Guo, "Deep interest with hierarchical attention network for click-through rate prediction," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2020, pp. 1905–1908.
[20] X. Zhang, Z. Wang, B. Du, J. Wu, X. Zhang, and E. Meng, "Deep session heterogeneity-aware network for click through rate prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 7927–7939, Dec. 2024.
[21] H. Guo, J. Yu, Q. Liu, R. Tang, and Y. Zhang, "PAL: A position-bias aware learning framework for CTR prediction in live recommender systems," in *Proc. 13th ACM Conf. Recommender Syst.*, Sep. 2019, pp. 452–456.
[22] M. Shirokikh, I. Shenbin, A. Alekseev, A. Volodkevich, A. Vasilev, A. V. Savchenko, and S. Nikolenko, "Neural click models for recommender systems," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2024, pp. 2553–2558.
[23] P. Hager, M. de Rijke, and O. Zoeter, "Contrasting neural click models and pointwise IPS rankers," in *Proc. Eur. Conf. Inf. Retr. (ECIR)*. Cham, Switzerland: Springer, 2023, pp. 409–425.
[24] O. A. S. Ibrahim and E. M. G. Younis, "Hybrid online–offline learning to rank using simulated annealing strategy based on dependent click model," *Knowl. Inf. Syst.*, vol. 64, no. 10, pp. 2833–2847, Oct. 2022, doi: 10.1007/s10115-022-01726-0.
[25] J. Kang, M. de Rijke, S. De Leon-Martinez, and H. Oosterhuis, "Rethinking click models in light of carousel interfaces: Theory-based categorization and design of click models," in *Proc. Int. ACM SIGIR Conf. Innov. Concepts Theories Inf. Retr. (ICTIR)*, Jul. 2025, pp. 44–55.
[26] L. Yan, Z. Qin, H. Zhuang, X. Wang, M. Bendersky, and M. Najork, "Revisiting two-tower models for unbiased learning to rank," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2022, pp. 2410–2414.
[27] R. M. Neal, "Markov chain sampling methods for Dirichlet process mixture models," *J. Comput. Graph. Statist.*, vol. 9, no. 2, pp. 249–265, Jun. 2000.
[28] D. M. Blei and M. I. Jordan, "Variational inference for Dirichlet process mixtures," *Bayesian Anal.*, vol. 1, no. 1, pp. 121–143, Mar. 2006.
[29] C. E. Rasmussen and Z. Ghahramani, "Infinite mixtures of Gaussian process experts," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 14, 2001, pp. 881–888.
[30] D. Görür and Y. W. Teh, "An efficient sequential Monte Carlo algorithm for coalescent clustering," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 21, 2008, pp. 521–528.
[31] Y. Li, E. Schofield, and M. Gönen, "A tutorial on Dirichlet process mixture modeling," *J. Math. Psychol.*, vol. 91, pp. 128–144, 2019.
[32] O. Chapelle and Y. Chang, "Yahoo! learning to rank challenge overview," in *Proc. Learn. Rank Challenge*, PMLR, 2011.
[33] X. Wang, N. Golbandi, M. Bendersky, D. Metzler, and M. Najork, "Position bias estimation for unbiased learning to rank in personal search," in *Proc. 11th ACM Int. Conf. Web Search Data Mining*, Feb. 2018, pp. 610–618.
[34] Q. Ai, T. Yang, H. Wang, and J. Mao, "Unbiased learning to rank: Online or offline?" *ACM Trans. Inf. Syst.*, vol. 39, no. 2, pp. 1–29, Apr. 2021.
[35] Z. Zhao, L. Hong, L. Wei, J. Chen, A. Nath, S. Andrews, A. Kumthekar, M. Sathiamoorthy, X. Yi, and E. Chi, "Recommending what video to watch next: A multitask ranking system," in *Proc. 13th ACM Conf. Recommender Syst.*, Sep. 2019, pp. 43–51.
[36] C. Xiong, X. Yu, W. Xu, L. Cheng, C. Yuan, and L. Mo, "A learnable fully interacted two-tower model for pre-ranking system," in *Proc. 48th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2025, pp. 2182–2191.
[37] Y. Wang, F. Xiong, Z. Han, Q. Song, K. Zhan, and B. Wang, "Unleashing the potential of two-tower models: Diffusion-based cross-interaction for large-scale matching," in *Proc. ACM Web Conf.*, Apr. 2025, pp. 304–312.
[38] G. Guo, Q. Wang, J. Allison, and G. Qian, "Accelerated distributed expectation-maximization algorithms for the parameter estimation in multivariate Gaussian mixture models," *Appl. Math. Model.*, vol. 137, Jan. 2025, Art. no. 115709.
[39] E. Serdyukov and W. Cukierski. (2013). *Personalized Web Search Challenge*. Kaggle Competition. [Online]. Available: https://www.kaggle.com/competitions/yandex-personalized-web-search-challenge

K. J. AMALA received the B.Tech. degree in computer science and engineering from Cochin University of Science and Technology and the M.E. degree in computer science from the Noorul Islam Centre for Higher Education. She is currently pursuing the Ph.D. degree in computer science and engineering in data science and business systems with the SRM Institute of Science and Technology, Tamil Nadu, India.

She has qualified for the University Grants Commission (UGC) National Eligibility Test (NET) and has over 12 years of teaching experience in undergraduate and postgraduate computer science education. She has working experience with large-scale web search datasets and dedicated to developing interpretable, adaptive, and user-centric information retrieval systems. Her research interests include learning to rank, machine learning, and artificial intelligence.

D. RAJESWARI received the B.Tech. degree in information technology from the Annai Mathammal Sheela Engineering College, Anna University, in 2008, the M.Tech. degree in information technology from the PSG College of Technology, Coimbatore, Anna University, in 2010, and the Ph.D. degree from the College of Engineering, Guindy, Anna University, in 2017, under the guidance of Dr. V. Jawahar Senthilkumar. From 2008 to 2010, she received a GATE stipend for completing M.Tech. degree. She received International Travel Support (ITS) from SERB to attend the 2023 POMS Annual Conference in USA, from 21 May to 25 May. She is currently a Professor with the Department of Data Science and Business Systems, School of Computing, College of Engineering and Technology, SRM Institute of Science and Technology, Kattankulathur, India.